REFERENCE ARCHITECTURE

Self-Hosted Enterprise Agentic Runtime Environment

Technical architecture for AI assistant agents that plan, call tools, run code, browse, remember, run routines, and work with company documents and databases, running entirely on company infrastructure with company-hosted open-weight models

Author	Balakrishna Vundavalli

Date	19 October 2026
Version	1.0 (draft for review)
Audience	Engineering leadership; platform, infrastructure, security and ML engineering teams
Status	For discussion. Sizing and cost figures are indicative estimates, not quotations
Classification	Internal

Document control

Version	Date	Author	Change
0.9	12 Oct 2026	B. Vundavalli	Initial structure and component analysis
1.0	19 Oct 2026	B. Vundavalli	Full draft: flows, threat model, roadmap, sizing

Conventions

- **MUST / SHOULD / MAY** are used in the RFC 2119 sense for architectural requirements.
- **Recommended** options are a starting default, not a mandate. Each component lists alternatives and trade-offs so teams can substitute existing company standards (for example an existing Kafka, Postgres or observability platform).
- Open-source projects and open-weight models change quickly. Validate versions, licences and maturity at selection time. Model names are examples of families, not fixed picks.
- All sizing and cost figures are **rough, assumption-based estimates** intended for budgeting conversations, and must be validated with load tests on the company's own hardware.

Contents

1. Executive Summary....5
1.1 Goals....5
1.2 Non-goals....6
1.3 Guiding principles....6
2. Requirements....7
2.1 Functional requirements....7
2.2 Non-functional requirements....7
3. High-Level Architecture....9
3.1 Layer responsibilities....10
3.2 Key architectural decisions....10
3.3 Request path in brief....10
4. Component Deep Dives....12
4.1 Model serving layer....12
4.2 Agent runtime and orchestrator....15
4.3 Tool layer using the Model Context Protocol (MCP)....17
4.4 Knowledge and retrieval-augmented generation (RAG)....19
4.5 Text-to-SQL and database access safety....20
4.6 Sandboxed execution environments....21
4.7 Memory....22
4.8 Routines and scheduler....23
4.9 Front ends and channels....23
4.10 Identity, authorisation and governance....24
4.11 Observability and evaluation....25
5. Key Sequence Flows....27
5.1 User chat turn with tool calls....27
5.2 RAG query with ACL filtering....28
5.3 Scheduled routine run....29
5.4 Approval-gated write action....30
6. Data Model Overview....31
7. Deployment Topology....33
7.1 Cluster layout and node pools....34
7.2 Networking....34
7.3 High availability....34
7.4 Disaster recovery....35
7.5 Environments....35
8. Security Threat Model....36
9. Phased Roadmap....37
10. Indicative Cost Considerations....38
10.1 Sizing assumptions....38
10.2 Indicative GPU counts by user scale....38
10.3 Hardware options....38
10.4 Cost components....39
11. Risks, Open Questions and Next Steps....40

11.1 Key risks....40
11.2 Open questions....40
11.3 Recommended next steps (next 30 to 60 days)....41
Appendix A. Recommended Open-Source Stack Summary....42
Appendix B. Glossary....43

1. Executive Summary

This document defines a vendor-neutral reference architecture for a self-hosted **agentic runtime environment**: a company platform on which employees converse with AI assistant agents that can reason over multi-step tasks, call tools, execute code in isolated sandboxes, browse the web or intranet, remember facts across sessions, run scheduled and event-driven routines, and work directly with company documents and databases.

The defining constraint is **sovereignty**. Every component, including the language models, runs on company-controlled infrastructure (on-premises data centres or a private-cloud tenancy under company control). No prompt, document, tool result or telemetry leaves the company boundary for an external model API. This is achieved by serving open-weight models on company GPUs behind an OpenAI-compatible internal gateway, and by connecting agents to internal systems through a governed tool layer built on the Model Context Protocol (MCP).

The architecture is organised into six layers:

1. **Channels**: a web chat UI, Slack/Teams bots, email, and a public API.
2. **Edge, identity and policy**: SSO, an API gateway and a central policy engine.
3. **Agent control plane**: an orchestrator running the agent loop, durable execution, memory, routines and human approvals.
4. **Capability layers**: model serving, an MCP tool layer and sandboxed execution.
5. **Data plane**: Postgres, vector/hybrid search, object storage, event bus and secrets vault.
6. **Observability, evaluation and audit** across all layers.

> **Headline recommendations**
>
> **Models**: serve one strong open-weight 'reasoning' model (a 70B-class dense model or a mid-size MoE) plus one small, fast model and an embedding/reranker pair, with **vLLM** (or SGLang) behind a **LiteLLM-style gateway**. Use FP8 or 4-bit quantisation to cut GPU count roughly in half.
>
> **Agent runtime**: a thin, custom agent loop (or LangGraph where graph-shaped workflows help) running as **Temporal** workflows, so long tasks, routines and approval waits survive restarts.
>
> **Tools**: everything external to the model is an **MCP server** registered in a versioned tool registry, classified read/write/destructive, and called with the **end user's delegated identity**, never a shared super-user.
>
> **Isolation**: per-session sandboxes in **Kata Containers or gVisor** on a dedicated Kubernetes node pool (Firecracker microVMs for higher assurance), with default-deny egress.
>
> **Safety**: tool output is treated as untrusted data; risky actions (sending, deleting, writing to production) require **human-in-the-loop approval** bound to the exact arguments; every action is written to a tamper-evident audit log.
>
> **Delivery**: a 10-week PoC with 3 to 5 engineers, reaching production in roughly 11 to 12 months with a team of 10 to 14.

1.1 Goals

#	Goal	What success looks like
G1	Full data sovereignty	Zero outbound calls to third-party model APIs; verified by egress policy and audit.
G2	Useful agents, not only chat	Agents complete multi-step tasks across tools (query, analyse, draft, file a ticket) with measurable task success rates.

#	Goal	What success looks like
G3	Secure by construction	Agents can never exceed the permissions of the user they act for; risky writes need approval; all actions are auditable.
G4	Grounded in company knowledge	Answers cite internal sources and respect document-level permissions.
G5	Extensible platform	Teams can add tools (MCP servers), agents and routines through self-service with review, without platform changes.
G6	Operable at enterprise scale	Multi-tenant, HA, DR, observable, cost-attributed per team; scales from pilot to thousands of users.
G7	Model agility	Swap or add models behind a stable API without changing agents or tools.

1.2 Non-goals

- **Training foundation models.** The platform consumes open-weight models; fine-tuning (for example LoRA adapters for text-to-SQL or domain style) is in scope later, pre-training is not.
- **Replacing systems of record.** Agents act on existing ERP, CRM, ticketing and document systems through their APIs; the platform does not become a new source of truth.
- **Fully autonomous high-risk actions.** Unattended agents performing irreversible or production-impacting actions without human approval are explicitly out of scope for initial phases.
- **General-purpose public chatbot.** The platform serves authenticated employees and approved service integrations, not anonymous external users.
- **Frontier-parity guarantees.** Open-weight models are highly capable but may trail the best proprietary models on some tasks; the design mitigates this with tools, retrieval, evaluation and routing rather than promising parity.
- **A low-code workflow builder in Phase 0 and 1.** Routines are defined in natural language plus configuration; a visual builder is a possible Phase 3 addition.

1.3 Guiding principles

- **Least privilege, delegated identity**: the agent is the user's assistant, never a privileged service account.
- **Model output is a proposal; tool output is data**: neither is trusted to authorise anything.
- **Everything durable**: any step that can take longer than a request timeout runs in a durable workflow.
- **Standard interfaces**: OpenAI-compatible inference API, MCP for tools, OIDC for identity, OpenTelemetry for telemetry, S3 API for objects.
- **Measure before you scale**: every change to prompts, models or tools runs through an evaluation suite.

2. Requirements

2.1 Functional requirements

ID	Capability	Requirement
F1	Conversational chat	Streaming multi-turn chat with Markdown, code, tables and inline artefacts; edit and regenerate; conversation history, search, sharing within ACLs; attachments (PDF, Office, images, CSV).
F2	Agents and multi-agent	Configurable agents (instructions, model, tools, knowledge, policies). An agent can delegate subtasks to subagents (e.g. researcher, coder, reviewer) with bounded budgets and scoped tools; results return to the parent.
F3	Tool calling	Structured function calling against a governed tool catalogue (MCP); parallel tool calls; retries; tool-result streaming; per-agent tool allowlists.
F4	Knowledge / RAG	Search over SharePoint, Confluence, file shares, wikis, tickets and uploaded files with hybrid retrieval, reranking, citations and document-level ACL enforcement.
F5	Database access	Natural-language analytics over Postgres, SQL Server, Oracle, Snowflake and others, read-only by default, with schema-aware text-to-SQL and safety guards.
F6	Code execution	Run Python (and optionally Node, shell) in an isolated sandbox with common data libraries; persistent per-session workspace; produce files and charts.
F7	Browser automation	Headless browser for intranet and allowlisted internet sites: navigate, read, fill forms (approval-gated for submissions), take screenshots.
F8	Memory	Session state; long-term per-user and per-agent memory (preferences, facts), team/shared memory; user-visible and editable; retention controls.
F9	Routines	Scheduled (cron) and event-driven runs (webhook, Slack message, email, DB change, file arrival) executing agent instructions with delivery to a channel.
F10	File generation	Generate DOCX, XLSX, PPTX, PDF, CSV, charts and images from sandbox code; store as artefacts with download links and retention.
F11	Integrations and channels	Slack and Microsoft Teams bots (DMs, mentions, threads, approval buttons), email in/out, REST/SSE API and SDK, embeddable widget.
F12	Human-in-the-loop	Approval requests for risky actions, with exact-argument previews, delegation of approvers, timeouts and audit.
F13	Administration	Agent, tool, model and policy management; quotas; usage dashboards; tenant and team administration; kill switches per tool, agent and model.

2.2 Non-functional requirements

Area	Requirement (initial targets, to be confirmed)
Data residency	All processing and storage on company infrastructure in approved locations (for Indian operations, consider DPDP Act 2023 obligations and any sector rules such as RBI or SEBI guidance). No external model APIs. Internet egress only via proxy allowlist; model weights mirrored internally.
Security	Zero-trust networking (mTLS between services), encryption in transit and at rest, secrets in Vault, sandbox isolation stronger than plain containers, prompt-injection defences, signed artefacts and images, regular red-teaming.
Multi-tenancy	Logical isolation by tenant (business unit / subsidiary) and team: separate data partitions, quotas, policies and keys; optional hard isolation (separate namespaces or clusters) for regulated tenants.
Scalability	Pilot 100 to 300 users; production 1,000 to 5,000; design headroom to 20,000+. Horizontal scaling of stateless services; GPU pools scale by replica.

Area	Requirement (initial targets, to be confirmed)
Latency	Time-to-first-token p50 under 1.5 s and p95 under 4 s for the chat model at target load; simple RAG answer end-to-end p95 under 10 s; tool call overhead under 300 ms excluding the tool itself.
Throughput	Size for peak concurrent active sessions of roughly 3 to 5 percent of named users, each agent turn averaging 3 to 10 model calls (assumption, refine from pilot telemetry).
Availability	Chat and API 99.5 percent in pilot, 99.9 percent in production (business hours critical); routines at-least-once with idempotency; graceful degradation to small model if large pool is unavailable.
Durability / DR	RPO 15 minutes or better for conversations and audit; RTO 4 hours for full service, 1 hour for chat with reduced capacity.
Cost	Per-team cost attribution (GPU-seconds and tokens); quotas and budgets; target high GPU utilisation through batching, quantisation and routing to small models.
Auditability	Immutable, tamper-evident record of every prompt, tool call, approval, data access and admin change, retained per policy (e.g. 1 to 7 years), exportable to the SIEM.
Privacy	PII detection and redaction in logs and traces; data-minimisation; right-to-erasure workflow covering memory and conversations.
Operability	GitOps deployment, infrastructure as code, SLO-based alerting, runbooks; zero-downtime upgrades of services and models (blue/green).
Accessibility / i18n	WCAG 2.1 AA web UI; multilingual support (English plus Indian languages as required, subject to model capability).

3. High-Level Architecture

Figure 1 shows the logical components grouped into layers. Requests enter through a channel, are authenticated at the edge, and are handled by the agent control plane. The orchestrator is the only component that talks to the model gateway, the tool layer and the sandbox manager; it consults the policy engine before every tool call. All capability layers read and write through the data plane, and every layer emits telemetry and audit events.

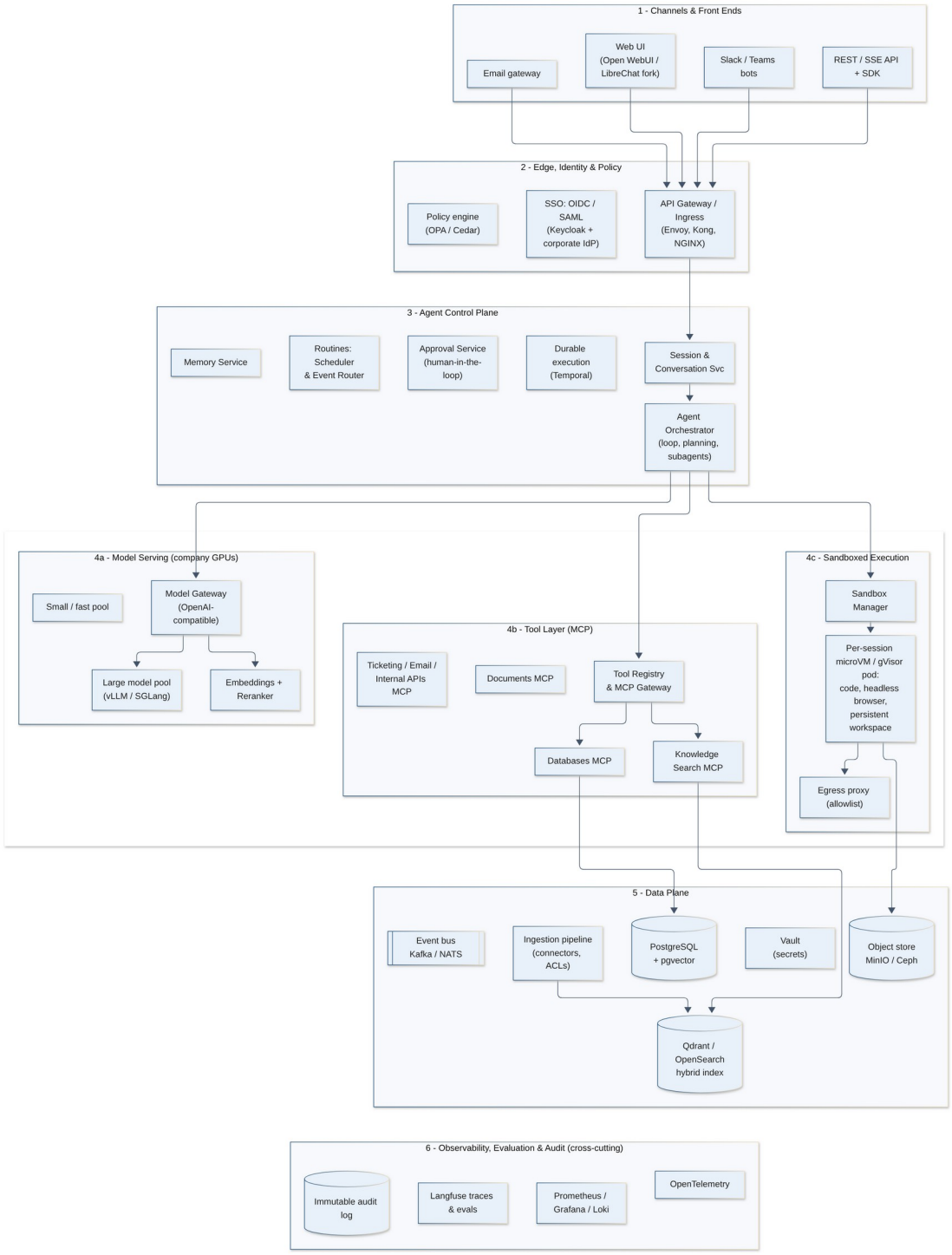


Figure 1: Logical component architecture (layered view)

3.1 Layer responsibilities

Layer	Key components	Responsibility
Channels	Web UI, Slack/Teams bots, email gateway, REST/SSE API	User interaction, streaming rendering, artefact display, approval cards; no business logic.
Edge and identity	API gateway, Keycloak / corporate IdP, OPA or Cedar	Authentication, token exchange, rate limiting, WAF; central authorisation decisions for tools, data and admin.
Agent control plane	Session service, Orchestrator, Temporal, Approval, Routines, Memory	Run agent loops; manage context; checkpoint long-running work; schedule routines; pause for approvals; extract and recall memory.
Model serving	Model gateway, vLLM/SGLang pools, embeddings/rerankers	OpenAI-compatible inference, routing, quotas, fallbacks, metering.
Tool layer	Tool registry, MCP gateway, MCP servers	Discoverable, versioned, permissioned tools for databases, documents, ticketing, email and internal APIs.
Sandboxed execution	Sandbox manager, microVM / gVisor pods, egress proxy	Isolated code execution, headless browsing, persistent workspaces, file generation.
Data plane	Postgres + pgvector, Qdrant/OpenSearch, MinIO/Ceph, Kafka/NATS, Vault, ingestion	State, search indexes, artefacts, events, secrets, connector-driven ingestion.
Observability and audit	OpenTelemetry, Langfuse, Prometheus, Grafana, Loki, audit store	Traces of every agent step, metrics, logs, evaluations, cost, tamper-evident audit.

3.2 Key architectural decisions

Decision	Choice	Rationale
Inference API	OpenAI-compatible chat/completions and embeddings via an internal gateway	De facto standard; supported by vLLM, SGLang, TGI, every agent framework and most UIs; allows model swaps without code changes.
Tool protocol	MCP (streamable HTTP transport) for all tools	Open standard with broad ecosystem support; decouples tool authors from the agent runtime; enables a single governance point.
Execution model	Agent turns and routines as durable workflows (Temporal)	Survives pod restarts and deployments; native timers for schedules; signals for approvals; full history for audit.
Isolation	Sandboxes on a dedicated node pool with a VM-grade runtime (Kata/Firecracker) or gVisor	Model-generated code is untrusted; shared-kernel containers alone are insufficient.
Identity	Agents act with the user's delegated, short-lived credentials	Prevents privilege escalation via the agent and preserves source-system audit trails.
State store	PostgreSQL as primary system of record (with pgvector for memory)	Mature, HA-capable, operationally familiar; reduces the number of stateful systems.
Deployment	Kubernetes with GitOps on company hardware or private cloud	Uniform scheduling of CPU and GPU workloads; standard tooling for HA, scaling and policy.

3.3 Request path in brief

1. A user sends a message from the web UI or Slack. The channel adapter calls the API with the user's OIDC token (or a mapped identity for Slack/Teams).

2. The session service persists the message and asks the orchestrator to start a turn, which runs as a Temporal workflow keyed by conversation.
3. The orchestrator assembles context (agent instructions, conversation summary, recent messages, recalled memories, tool schemas filtered by policy) and calls the model gateway.
4. When the model emits tool calls, the orchestrator checks policy, obtains delegated credentials, invokes the MCP server or sandbox, wraps the result as untrusted data, and loops.
5. Risky calls pause the workflow and create an approval request. The final answer streams back to the channel; messages, tool calls, usage and audit events are recorded.

4. Component Deep Dives

Each subsection describes responsibilities, recommended technology options and the main trade-offs. Recommendations favour mature, permissively licensed open-source projects that run fully offline.

4.1 Model serving layer

Responsibilities

- Host open-weight LLMs, vision-language models, embedding models and rerankers on company GPUs.
- Expose one **OpenAI-compatible** endpoint (chat, completions, embeddings, rerank) with model aliases, routing, quotas, fallbacks and metering.
- Provide reliable structured output and tool calling (JSON schema-constrained decoding, model-specific tool-call parsers).
- Support zero-downtime model upgrades, A/B and shadow traffic, and per-tenant model access policies.

Inference engines

Engine	Strengths	Trade-offs	Recommendation
vLLM	PagedAttention, continuous batching, prefix caching, tensor/pipeline/expert parallelism, wide model coverage, OpenAI-compatible server, tool-call parsers, guided decoding, LoRA multiplexing, NVIDIA and AMD support.	Fast-moving releases; tuning flags matter; some features model-specific.	**Default** for LLMs.
SGLang	Very high throughput, RadixAttention prefix reuse (excellent for agent loops with repeated system prompts), strong MoE and DeepSeek-family support, structured output.	Smaller operator community than vLLM; fewer turnkey integrations.	**Strong alternative**; benchmark both on your workload.
Hugging Face TGI	Simple deployment, production heritage, good HF integration.	Development pace has slowed relative to vLLM/SGLang; check project status before adopting.	Acceptable if already standard in-house.
NVIDIA Triton + TensorRT-LLM	Highest per-GPU performance on NVIDIA, multi-model serving, ensembles; good for embeddings/rerankers and classic ML.	Engine build step per model and GPU type; NVIDIA-only; more operational complexity.	Use for embeddings/rerankers or when squeezing maximum throughput.
Hugging Face TEI / Infinity	Purpose-built embedding and reranker servers, very efficient on small GPUs or even CPU.	Embeddings and rerankers only.	**Default** for embeddings/rerankers.
llama.cpp / Ollama	Runs on CPU or small GPUs; trivial setup.	Not designed for high-concurrency multi-tenant serving.	Developer laptops and edge only.

Open-weight model families

The open-weight landscape moves every few months. Select by evaluated performance on the company's own agentic task suite (tool-call accuracy, multi-turn reliability, instruction following, multilingual needs), licence and hardware fit, not by leaderboards alone. As of October 2026, families worth shortlisting include:

Family (examples)	Typical role	Notes
Qwen (Qwen3 / 3.5 / 3.6 series, dense and MoE, Coder variants)	Primary reasoning and tool-use model; small fast model; coder	Apache 2.0 for most sizes; strong tool calling and multilingual; wide size range from ~9B to several hundred billion parameters (MoE).
Llama (Llama 3.x 70B, Llama 4 MoE)	General assistant	Mature ecosystem; community licence with acceptable-use and scale conditions, so review with Legal.
Mistral (Mistral Small, Ministral, Mistral Large)	Efficient general and tool-use models	Several Apache 2.0 releases; good function calling; European vendor.
DeepSeek (V3.x / V4, R1-style reasoning)	Top-end reasoning and coding	MIT-licensed weights; very large MoE models need multi-node or 8x high-memory GPU nodes.
OpenAI gpt-oss (20B, 120B)	Efficient tool-use and reasoning	Apache 2.0; MoE with low active parameters; 120B fits on a single 80 GB GPU in native MXFP4.
Others: GLM, Gemma, IBM Granite, Phi	Small models, classification, guardrails, on-CPU tasks	Useful for routing, PII detection, summarisation and safety classifiers.
Embeddings: BGE-M3, Qwen3-Embedding, multilingual-e5, nomic-embed	Retrieval	BGE-M3 offers dense + sparse in one model and strong multilingual coverage.
Rerankers: bge-reranker-v2-m3, Qwen3-Reranker, mxbai-rerank	Precision for RAG	Cross-encoders; run on small GPUs; large gains in answer quality per unit cost.
Guardrail classifiers: Llama Guard, Granite Guardian, ShieldGemma	Input/output safety screening	Small models run alongside the main pool.

Recommended starting portfolio

Reasoning tier: one model in the ~70B-dense or ~100-250B MoE class with proven tool calling (e.g. a Qwen 3.x MoE, gpt-oss-120b or Llama 3.3 70B) in FP8.

Fast tier: one ~7B to 32B model (or a small-active MoE such as a ~30B-A3B) for routing, summarisation, memory extraction, titles and classification.

Retrieval: BGE-M3 (or Qwen3-Embedding) plus a cross-encoder reranker on TEI.

Vision (Phase 1+): a vision-language model for screenshots, scanned documents and charts.

Keep weights in an internal model registry (e.g. an S3 bucket plus MLflow, or a Hugging Face Hub mirror) with checksums, licence metadata and evaluation results; nodes never pull from the internet.

GPU sizing guidance

Two quantities determine GPU memory: **weights** (parameters x bytes per parameter) and **KV cache** (grows with concurrent tokens in flight). Compute throughput determines tokens per second. For MoE models, all experts must be resident in memory, but per-token compute scales with **active** parameters, which makes MoE attractive for throughput.

Precision	Bytes / param	70B dense weights	Quality impact (typical)
BF16 / FP16	2	~140 GB	Reference
FP8 (H100/H200/B200, MI300-class)	1	~70 GB	Negligible for most tasks; **recommended default**
INT8 (W8A8 / SmoothQuant)	1	~70 GB	Small; for GPUs without FP8 (A100, L4 class)

Precision	Bytes / param	70B dense weights	Quality impact (typical)
4-bit (AWQ, GPTQ, MXFP4, NVFP4)	~0.5	~35 to 40 GB	Small to moderate; validate tool-call accuracy carefully

Worked example (illustrative). A 70B-class dense model with grouped-query attention (80 layers, 8 KV heads, head dimension 128) needs about 2 x 80 x 8 x 128 x 2 bytes = ~0.33 MB of FP16 KV cache per token (~0.16 MB with FP8 KV cache). Serving 32 concurrent sequences averaging 16k tokens of context needs ~512k tokens of cache, i.e. ~170 GB in FP16 or ~85 GB in FP8. With FP8 weights (~70 GB) plus FP8 KV cache, a 4 x 80 GB GPU replica (tensor parallel 4) leaves comfortable headroom; a 2 x 80 GB replica works for lower concurrency. Agent workloads have long, repetitive prompts (system prompt plus tool schemas), so **prefix caching** substantially reduces effective prefill cost.

Model class	Example deployment per replica	Notes
7B to 14B dense	1 x L40S / L4 (48 / 24 GB) or 1 x A100/H100 shared via MIG	Fast tier; very high throughput.
27B to 35B dense or ~30B small-active MoE	1 x 80 GB GPU (FP8), or 2 x 48 GB	Good fast-to-mid tier; strong cost-performance.
70B dense	2 to 4 x 80 GB GPUs (FP8), TP 2 to 4	Common reasoning tier.
100 to 250B MoE	1 to 4 x 80 GB, or 2 to 4 x 141 GB / 192 GB GPUs depending on quantisation	Throughput close to a much smaller dense model.
600B+ MoE (frontier-class)	Full 8-GPU node of 141 GB+ GPUs, or multi-node with expert parallelism	Highest quality; highest capex; consider only after Phase 2 evidence.
Embeddings + reranker	1 small GPU (L4 / L40S) or a MIG slice; CPU possible at low volume	Batch ingestion separately from query-time traffic.

Always validate with load tests on the target hardware (for example vLLM's benchmark scripts or GuideLLM) using recorded agent traffic shapes: long prefills, short outputs, many sequential calls per turn.

Model gateway and routing

A gateway (LiteLLM proxy is the reference open-source option; Envoy AI Gateway, Kong AI plugins or a thin in-house proxy are alternatives) sits in front of all engines and provides:

- **Model aliases** (e.g. `reasoning`, `fast`, `vision`, `embed`, `rerank`) so agents never hard-code model names, enabling blue/green upgrades.
- **Virtual keys per agent, team and tenant** with rate limits, token budgets and model access lists.
- **Routing and fallbacks**: least-busy or latency-based load balancing across replicas; fallback to a secondary replica or the fast tier on overload; circuit breakers.
- **Metering**: tokens in/out, cache hits, latency and GPU-seconds per request, exported to the cost and observability stack.
- **Guardrail hooks**: optional pre/post calls to PII and safety classifiers.
- **Strict 'no external provider' configuration**: only internal endpoints are registered; egress from the gateway namespace is denied at the network layer.

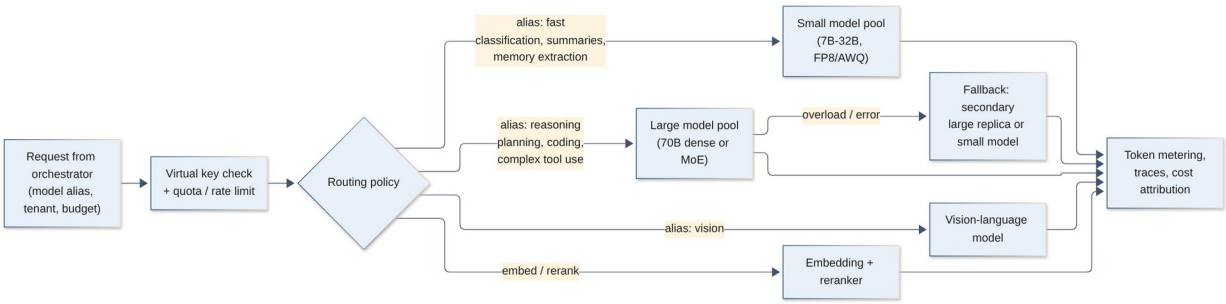


Figure 2: Model gateway routing between small, large, vision and retrieval models

Routing policy: start with explicit routing (the orchestrator chooses the alias per step: planning and tool selection on `reasoning`; summarisation, memory extraction, titles and classification on `fast`). Add learned or heuristic routing (difficulty classifier) only after evaluation data shows savings without quality loss. Speculative decoding (draft model or EAGLE-style heads) can cut latency for the reasoning tier where supported.

Trade-offs

- **One large model vs many specialised**: start with one reasoning and one fast model to limit operational burden; add specialists (coder, vision, SQL-tuned LoRA) when evaluations justify them.
- **Quantisation**: FP8 is close to free; 4-bit halves memory again but must be validated on tool-calling and SQL tasks, where small accuracy losses compound across steps.
- **Dedicated vs shared GPU pools**: dedicated pools per tenant give isolation and predictable latency but lower utilisation; prefer shared pools with quotas, plus dedicated pools only for regulated tenants.
- **Long context**: large windows (128k+) are available but costly in KV cache; prefer retrieval and summarisation over stuffing context.

4.2 Agent runtime and orchestrator

Responsibilities

- Execute the **agent loop**: build context, call the model, parse tool calls, authorise, execute, observe, repeat until a final answer, a budget limit or a pause.
- Manage **planning** (explicit plan or to-do list for long tasks), **subagents/delegation**, **context window** budgeting and summarisation, retries and error recovery.
- Enforce per-turn **budgets**: maximum steps, tokens, wall-clock time, cost and tool calls.
- Stream partial output and step events (thinking status, tool started/finished, artefacts) to channels.
- Persist every step for replay, debugging, evaluation and audit.

The agent loop

```
turn(conversation, user_msg):
  ctx = assemble(agent.instructions, policy_summary,
                 memory.recall(user, user_msg), conversation.summary,
                 recent_messages, tool_schemas(filtered_by_policy))
  for step in 1..MAX_STEPS:
    resp = gateway.chat(alias=pick_alias(step), messages=ctx,
                        tools=schemas, stream=True)
    if resp.final_answer:
      stream_to_user(resp); break
    for call in resp.tool_calls:              # parallel if independent
      decision = policy.authorise(user, agent, call)
      if decision == DENY: ctx.append(denied(call)); continue
      if decision == REQUIRE_APPROVAL:
        await approval(call)                  # durable wait (signal)
      creds  = vault.delegated(user, call.tool)
      result = execute(call, creds)
      ctx.append(tool_result(wrap_untrusted(truncate(result))))
    if ctx.tokens > 0.7 * BUDGET:
      ctx = compact(ctx)                      # summarise older steps
```

```
    checkpoint(step)                          # durable history
persist(); memory.extract_async(conversation)
```

Planning, subagents and delegation

- **Implicit planning** (model decides next step each iteration) is sufficient for most chat turns. For long tasks the agent maintains an explicit **plan/to-do artefact** in its workspace that it updates as it goes; this improves reliability with open-weight models and gives users visibility.
- **Subagents** are spawned via a `delegate` tool: the parent passes a task brief, a tool subset, a model alias and a budget. The subagent runs as a **child workflow** with its own context window and returns a concise result. This isolates context (the parent never sees thousands of tokens of raw search results), allows parallelism and narrows permissions.
- Common patterns: **researcher** (search and read, read-only tools), **analyst/coder** (sandbox and SQL), **reviewer/critic** (checks output against requirements before delivery), **router** agents that pick a specialist.
- Guardrails: maximum delegation depth (e.g. 2), total budget inherited from the parent, subagents cannot obtain tools the parent lacks, and the user's identity propagates unchanged.

Context window management

Technique	Description
Token budgeting	Reserve fixed budgets for system prompt, tool schemas, memory, retrieved knowledge, history and output; enforce before each call.
Tool schema filtering	Expose only tools relevant to the agent and, for large catalogues, retrieve the top-k tools by semantic match to the task (tool search) rather than loading hundreds of schemas.
Result truncation and offloading	Large tool results (query results, web pages, files) are stored in the workspace or object store; the model sees a preview plus a handle it can page through.
Rolling summarisation (compaction)	When history exceeds ~70 percent of the window, the fast model summarises older turns into a structured summary (decisions, facts, open items, file handles). Raw history stays in the database.
Prefix stability	Keep system prompt and tool schemas byte-stable across calls to maximise prefix-cache hits in vLLM/SGLang.
Scratchpad files	Agents write intermediate notes to the workspace instead of holding them in context.

Framework choice: LangGraph vs custom loop

Option	Pros	Cons	Fit
Custom loop (thin, in-house)	Full control of context, streaming, policy hooks and durability; minimal dependencies; easy to reason about and audit.	You own the code: retries, parallel tools, subagents and tracing must be built.	**Recommended core** (a few thousand lines), using the OpenAI-compatible client and MCP SDK directly.
LangGraph	Explicit graph/state-machine modelling, checkpointers, human-in-the-loop interrupts, good for deterministic multi-step workflows.	Abstraction overhead; its own persistence model overlaps with Temporal; version churn.	Good for **specific structured workflows** (e.g. a fixed document-review pipeline) running inside an activity.

Option	Pros	Cons	Fit
Other frameworks (OpenAI Agents SDK with local endpoints, PydanticAI, LlamaIndex agents, Microsoft Agent Framework / Semantic Kernel, CrewAI)	Fast to prototype; useful building blocks.	Opinionated; may assume hosted-model features; varied maturity.	Prototyping; selectively reuse components.

Durable execution

Agent turns can last minutes (and routines or approval waits, hours or days). Plain request/response services lose this work on restarts. **Temporal** (MIT-licensed, self-hostable on Postgres or Cassandra) is recommended:

- Each conversation turn, routine run and subagent is a **workflow**; model calls, tool calls and sandbox executions are **activities** with timeouts, retries and heartbeats.
- **Signals** deliver approvals, user interrupts ('stop', 'also do X') and Slack replies to a running workflow; **timers** implement waits and schedules.
- The workflow history provides an exact replayable record of the run, complementing the audit log.
- Idempotency keys on write tools ensure at-least-once activity retries do not duplicate side effects.

Alternatives: a queue-based design (Kafka/NATS JetStream/Redis Streams plus a state table with step checkpoints) is simpler to start but requires hand-built retries, timers and resumption; Restate, DBOS or Hatchet are lighter durable-execution options; Argo Workflows suits batch pipelines rather than interactive turns. For Phase 0, an in-process loop with Postgres checkpoints is acceptable, with Temporal adopted from Phase 1.

4.3 Tool layer using the Model Context Protocol (MCP)

MCP is an open protocol that standardises how an AI application discovers and invokes **tools**, reads **resources** and uses **prompts** exposed by servers. In this architecture, every capability outside the model (except the core sandbox, which may also be exposed via MCP) is an MCP server, which gives one integration contract, one security choke point and a reusable ecosystem.

Topology

- **MCP servers run as internal services** (streamable HTTP transport) in Kubernetes, owned by the platform team or by the team owning the underlying system. Local stdio servers are only used inside sandboxes.
- An **MCP gateway** (in-house service, or an open-source MCP gateway/proxy) fronts all servers: authenticates the orchestrator, injects the user's delegated credentials, applies policy, rate limits, redacts, logs and meters each call.
- The orchestrator sees a **virtual, per-user tool catalogue** assembled from the registry: only tools the agent is configured for and the user is entitled to.
- Third-party or community MCP servers are **vendored, reviewed, pinned and rebuilt** internally; they never run straight from public registries.

Recommended initial MCP servers

Domain	MCP server(s)	Typical tools	Default class
Relational databases	Postgres, SQL Server, Oracle, MySQL; one server per engine with a shared safety core (see 4.5)	list_schemas, describe_table, sample_rows, run_query (read), explain	Read

Domain	MCP server(s)	Typical tools	Default class
Data warehouse	Snowflake, Databricks/Spark SQL, BigQuery-on-prem equivalents, Trino	semantic_search_metrics, run_query with warehouse cost guard	Read
Documents	SharePoint/OneDrive (Microsoft Graph), Confluence, Jira, SMB/NFS file shares, Google Drive if used	search, get_document, list_folder, create_page (write)	Read / Write
Knowledge search	Platform RAG service (4.4)	knowledge.search, knowledge.get_chunk, knowledge.list_collections	Read
Ticketing / ITSM	Jira, ServiceNow, GitLab/GitHub Enterprise issues	search_issues, get_issue, create_issue, comment, transition	Read / Write
Communication	Email (Exchange/Graph, IMAP/SMTP), Slack, Teams, calendar	search_mail, read_thread, draft_message, send_message, find_free_time	Read / Write (send = approval)
Internal APIs	Generated from OpenAPI specs (HR, CRM, ERP, finance), wrapped with explicit tool descriptions	get_employee, get_customer, create_purchase_request	Varies
Sandbox and browser	Sandbox manager (4.6)	run_code, read_file, write_file, browser.navigate, browser.screenshot	Read / Write (scoped to workspace)

Tool registry and versioning

- Registry entry per tool: name, owning team, MCP server and version, JSON schema, natural-language description, **risk class**, required scopes, data classification touched, rate limits, cost hints, approval policy, test suite reference and status (draft, approved, deprecated).
- **Semantic versioning** of servers; tool schemas are immutable per version. Agents pin a major version; minor upgrades roll out progressively with evaluation gates.
- **Description quality is a security and quality control**: descriptions are reviewed (they are effectively prompts) and diffed on change; changes in a description trigger re-evaluation, defending against 'tool poisoning' where a server alters instructions after approval.
- **Kill switches** at tool, server and agent level, effective within seconds.

Read vs write tools

Risk class	Examples	Controls
R0: Read, low sensitivity	Search public intranet, describe schema	Allowed by default per agent configuration.
R1: Read, sensitive	HR records, customer PII, financial data	Entitlement check, ABAC on data classification, PII redaction in traces, purpose logging.
W1: Reversible write, own scope	Create draft, add comment, write to own workspace	Allowed with audit; idempotency keys; undo where possible.
W2: External or shared side effects	Send email/Slack, create ticket in another team's queue, update CRM	**Human approval** of exact arguments (can be pre-approved per routine).
W3: Destructive or production-impacting	Delete records, DDL/DML on production, payments, permission changes	Approval plus step-up authentication, optional four-eyes rule, dry run and diff preview, change-window policy; disabled by default.

Prefer **narrow, intention-revealing tools** (e.g. `create_jira_issue(project, summary, description)`) over generic ones (`http_request`, `execute_sql_write`). Narrow tools are easier to authorise, approve, test and explain to users.

4.4 Knowledge and retrieval-augmented generation (RAG)

Ingestion pipeline

1. **Connectors** pull or receive change notifications from SharePoint (Microsoft Graph delta queries), Confluence, file shares, ticketing, wikis and uploaded files. Each connector records source ID, URL, version, last-modified and the **permission set** (users and groups with read access).
2. **Parsing and normalisation**: layout-aware parsing for PDF, Office, HTML and scans (e.g. Docling, Apache Tika, Unstructured; OCR via Tesseract or PaddleOCR; a vision-language model for complex tables and figures). Output is clean Markdown with structural markers (headings, tables, page numbers).
3. **Chunking**: structure-aware chunks of roughly 300 to 800 tokens with 10 to 15 percent overlap, never splitting tables mid-row; each chunk carries its heading path, and optionally a short model-generated context sentence ('contextual chunking') which measurably improves retrieval.
4. **Enrichment**: language detection, PII and classification tags, entity extraction, document-level summary for coarse retrieval.
5. **Embedding** in batches on the embedding pool (separate from query-time traffic), plus sparse vectors or BM25 indexing.
6. **Indexing** with metadata: tenant, collection, source, ACL principals, classification, timestamps, version hash.

Ingestion runs as idempotent, resumable jobs (Temporal or Argo Workflows), keyed by source document version hash so unchanged documents are skipped.

Vector and hybrid search options

Store	Strengths	Trade-offs	Fit
pgvector (Postgres)	No new system; transactional with app data; HNSW indexes; row-level security; good to tens of millions of vectors.	Hybrid search requires combining with Postgres full-text; scaling beyond ~50 to 100M vectors needs care.	Memory store; Phase 0/1 RAG; smaller corpora.
Qdrant	Fast, filter-aware HNSW (payload filtering performs well with ACL filters), sparse+dense hybrid, quantisation, simple to run.	Another stateful cluster to operate.	**Recommended** dedicated RAG store.
OpenSearch	Mature BM25 + k-NN hybrid in one engine, aggregations, document-level security plugin, familiar to many ops teams.	Heavier; vector performance below specialised engines at large scale.	**Recommended** if OpenSearch/Elasticsearch already runs in-house.
Milvus	Designed for billions of vectors, GPU indexes, disaggregated storage.	Most complex to operate (multiple components, etcd, object store, message queue).	Very large corpora (Phase 3).
Weaviate, Vespa	Hybrid search and rich ranking (Vespa especially).	Smaller in-house skill base in many companies.	Alternatives.

Retrieval pipeline

- **Query understanding**: the agent issues explicit `knowledge.search` calls (agentic RAG) and can reformulate, decompose or iterate; optional query expansion on the fast model.

- **Hybrid retrieval**: dense vector plus BM25/sparse, fused with reciprocal rank fusion; top 30 to 100 candidates.
- **Reranking** with a cross-encoder to top 5 to 10; optional diversity (maximal marginal relevance) and recency boosts.
- **Citations**: every chunk returns title, URL, version and modified date; the UI renders source links and the agent is instructed to cite.
- **Answerability**: if top scores are below threshold the agent says it did not find an answer rather than guessing.

Document-level ACL enforcement

Permissions are the hardest part of enterprise RAG. The design uses **early binding with late verification**:

- **Early binding**: each chunk stores the list of principals (user IDs and group IDs) allowed to read it, copied from the source at ingestion. At query time the knowledge service resolves the caller's principals (user plus transitive group memberships from the IdP/directory, cached briefly) and applies them as a **mandatory filter inside the vector query**, so unauthorised chunks are never retrieved, not merely hidden afterwards.
- **Late verification** for high-sensitivity collections: re-check permissions against the source system (e.g. Graph API) for the final handful of documents before returning them.
- **Permission freshness**: subscribe to permission-change events or run frequent delta crawls of ACLs (more often than content), and propagate group-membership changes; target ACL staleness under 15 minutes for sensitive sources.
- The filter is applied by the knowledge service using the identity from the **verified token**, never from model-supplied arguments, so prompt injection cannot widen access.
- Uploaded files are scoped to the conversation or user unless explicitly shared.

Freshness and re-indexing

- Incremental sync via webhooks/delta APIs where available; scheduled crawls elsewhere (hourly for active sources, daily for archives).
- Deletions propagate as tombstones within the same SLA as permission changes.
- **Re-embedding** when the embedding model changes: build a new index version in parallel (blue/green collection alias), backfill, evaluate retrieval quality, switch alias, then retire the old index.
- Per-collection dashboards: document counts, lag, failures, last successful sync.

4.5 Text-to-SQL and database access safety

Database access is high value and high risk: a plausible but wrong query misleads decisions, and an unsafe query can leak or damage data. Layered controls apply, so that no single control (least of all the model's own judgement) is relied upon.

Control	Implementation
Read-only by default	Connect to **read replicas**, reporting databases or warehouse shares, never primaries. Database roles have SELECT only on approved schemas; the connection is opened with a read-only transaction (e.g. `SET TRANSACTION READ ONLY` / `default_transaction_read_only`).
Delegated identity and RLS	Queries run under the end user's identity (Kerberos/OIDC pass-through, or a per-user mapped role) so existing **row-level security** and column masking apply (Postgres RLS, SQL Server RLS and dynamic data masking, Oracle VPD, Snowflake row access and masking policies).
Statement validation	Parse SQL with a real parser (e.g. sqlglot) before execution: allow only single SELECT/WITH statements; block DDL/DML, multiple statements, procedures with side effects, system catalogues outside allowlist, cross-database links and functions such as file or network access.

Control	Implementation
Allowlists	Schema/table/column allowlists per agent and data domain; optionally **parameterised query templates** or a semantic layer (dbt metrics, Cube) for governed KPIs, with free-form SQL only for approved analysts.
Result limits	Enforce `LIMIT` / `TOP` injection, maximum rows (e.g. 1,000 to the model, larger exports to the sandbox as files), maximum bytes, statement timeouts (e.g. 30 s), and warehouse cost or credit guards.
Workload protection	Dedicated resource pools/warehouses for agent traffic; concurrency limits per user; EXPLAIN-based cost check before running expensive queries.
Accuracy aids	Curated schema descriptions, column glossary and sample queries in a retrieval index; schema-linking step; self-check by EXPLAIN and dry run; show SQL to the user with results; evaluation set of question/SQL pairs per domain; optional SQL-specialised LoRA.
Writes	Separate, narrow write tools (stored procedures or API calls) under W2/W3 approval rules; never a general 'execute SQL write' tool against production.
Audit	Log SQL text, parameters, row counts, user, agent and conversation; feed to database activity monitoring.

4.6 Sandboxed execution environments

Agents execute model-written code and drive browsers; both must be assumed hostile. Sandboxes provide a disposable but optionally persistent computer per session or per agent.

Isolation options

Option	Isolation	Start-up	Notes
Standard containers (runc)	Shared kernel; namespaces and cgroups	Fast	**Not sufficient** on its own for untrusted code.
gVisor (runsc)	User-space kernel intercepts syscalls	Fast (sub-second)	Good default on Kubernetes via RuntimeClass; some syscall/performance overhead; limited GPU support.
Kata Containers	Lightweight VM per pod (QEMU, Cloud Hypervisor or Firecracker)	~1 s	**Recommended** for VM-grade isolation with Kubernetes ergonomics; needs nested virtualisation or bare metal.
Firecracker microVMs (direct)	Minimal VMM, strong isolation, snapshot/restore	~100 to 300 ms with snapshots	Highest density and fastest warm starts; requires building your own orchestration (or use an open-source sandbox platform built on it).

Sandbox design

- **Sandbox manager** service allocates sandboxes from a **warm pool**, binds them to a session, enforces quotas and reaps idle ones (e.g. suspend after 15 minutes idle, delete after a retention period).
- **Image**: minimal OS, Python with data libraries (pandas, numpy, matplotlib, openpyxl, python-docx, python-pptx, reportlab), Node, LibreOffice for document conversion, Playwright with headless Chromium, common CLI tools; built and signed in CI, scanned for CVEs; internal package mirrors (PyPI/npm via Nexus or Artifactory) instead of internet.
- **Persistent workspace**: a per-session or per-agent volume (PVC or object-store-backed filesystem) mounted at a fixed path; artefacts copied to the object store with signed download URLs; snapshot on suspend so sessions can resume.

- **Headless browser**: Playwright/Chromium inside the sandbox, controlled by browser tools (navigate, click, type, read DOM/accessibility tree, screenshot). Screenshots can feed a vision model. Submitting forms or logging into systems is a write action subject to approval.
- **Interface**: an in-sandbox agent daemon exposes run_code, file operations and browser control over an authenticated channel (gRPC or MCP); no SSH.

Controls

Control	Setting (initial)
Network egress	Default-deny NetworkPolicy; all HTTP(S) via an egress proxy with domain allowlists per agent/tenant (internal package mirror, approved intranet and internet sites); no access to cluster, metadata or control-plane addresses; DNS restricted.
Credentials	No long-lived secrets inside sandboxes. When code must call an internal API, use short-lived scoped tokens injected per call or a credential-injecting proxy, never visible to the model.
Resource limits	e.g. 2 vCPU / 4 GB default, 8 vCPU / 32 GB on request; disk quota 10 to 20 GB; process and file-descriptor limits; execution timeout per call (e.g. 120 s) and per session.
Kernel hardening	Seccomp and AppArmor/SELinux profiles, non-root user, read-only root filesystem except workspace, no privileged mode, no host mounts.
Node isolation	Dedicated tainted node pool for sandboxes; no co-location with control-plane or data services.
Data handling	Classification-aware: sandboxes processing restricted data get no internet egress at all; workspaces encrypted; retention and wiping per policy.
Monitoring	Falco/Tetragon runtime detection, egress logs, resource anomaly alerts, crypto-mining detection.

4.7 Memory

Memory type	Scope and content	Storage	Retrieval
Working / session state	Current turn context, plan, scratch notes, tool results, rolling summary	Temporal workflow state, Postgres (conversation), sandbox workspace	Always loaded (budgeted).
Conversation history	Full message and tool-call log	Postgres (partitioned by month), object store for large payloads	Search and 'recall past conversation' tool.
Long-term user memory	Preferences ('prefers tables', 'works in IST'), role, projects, recurring facts	Postgres + pgvector table of memory facts with scope, provenance, confidence and expiry	Top-k semantic + recency at turn start; explicit memory.search tool.
Agent memory	Lessons learned for an agent ('the finance DB uses fiscal year starting April')	Same store, scope = agent, curated	Loaded for that agent; changes reviewed by agent owners.
Shared / team memory	Team conventions, glossary, contacts	Same store, scope = team; or a knowledge collection	ACL-filtered like documents.

- **Write path**: after a conversation (asynchronously) the fast model proposes candidate facts; a policy filter drops secrets, credentials and sensitive personal data; duplicates are merged and contradictions supersede older facts (with history). Users can also say 'remember that…' explicitly.
- **User control**: a memory panel to view, edit and delete facts; per-user opt-out; deletion propagates to backups within the retention SLA (supports erasure requests).

- **Safety**: memories are data, not instructions. Facts derived from tool output (e.g. a web page) are marked lower-trust and are never promoted to agent-level memory without review, to prevent persistent prompt injection ('memory poisoning').
- **Alternatives**: open-source memory layers (e.g. Mem0, Letta, Zep/Graphiti for temporal knowledge graphs) can be evaluated; a Postgres-based implementation is recommended initially for simplicity, governance and ACL control.

4.8 Routines and scheduler

A **routine** is a saved agent task with a trigger, instructions, an agent configuration, a budget, a delivery target and an owner. Routines run with the **owner's delegated permissions** (or those of a designated service identity approved by governance) and are subject to the same policies as interactive turns.

Trigger	Mechanism	Examples
Cron / schedule	Temporal Schedules (time-zone aware, e.g. Asia/Kolkata), with jitter, catch-up and overlap policies	Weekly sales digest Monday 09:00 IST; nightly data-quality check.
Webhook	Authenticated endpoint (HMAC or mTLS) on the event router	CI failure, monitoring alert, form submission.
Slack / Teams	Bot events: message in channel, mention, reaction, slash command	Triage new messages in #support; summarise thread on reaction.
Email	Inbound mailbox monitored via Graph/IMAP	Classify and draft replies to vendor invoices.
Database change	CDC with Debezium into Kafka, or Postgres LISTEN/NOTIFY, filtered by predicate	New high-value order; status changed to 'escalated'.
File arrival	Object store bucket notifications, SharePoint folder delta	New contract PDF uploaded: extract key terms.

- **Event router** normalises events into a common envelope on the event bus, deduplicates by idempotency key, applies per-routine filters and rate limits, and starts a workflow.
- **Safety**: routines are reviewed when created or changed if they use W2/W3 tools; pre-approved delivery (e.g. 'post to #sales-digest') can be granted per routine; any other side effect requests approval from the owner. Routines are automatically paused when the owner leaves the company or loses entitlements.
- **Budget and loop protection**: per-run step/token/time limits; per-routine daily caps; circuit breaker if a routine fails or triggers itself repeatedly.
- **Observability**: run history with status, outputs, cost and trace link; failure notifications to the owner.

4.9 Front ends and channels

Channel	Approach	Notes
Web UI	Fork an open-source chat UI such as **Open WebUI** or **LibreChat** (both support OpenAI-compatible backends, SSO, file upload, artefacts), or build a React/Next.js UI on the platform API. Check each project's licence terms (e.g. branding clauses) before forking.	Forking accelerates Phase 0/1. For Phase 2, keep the UI thin and move agent logic, tools and policy into the platform so the UI is replaceable. Required features: streaming, step/tool timeline, citations, artefact viewer, approval cards, memory panel, agent and routine management.
Slack bot	Slack Bolt app (Socket Mode avoids inbound internet exposure)	DMs, mentions, thread context, interactive approval buttons, file uploads; map Slack user to corporate identity via SCIM/email; respect channel membership when reading.

Channel	Approach	Notes
Microsoft Teams bot	Bot Framework / Teams AI library, Adaptive Cards for approvals	Requires Azure Bot registration for Teams connectivity; message content still processed on company infrastructure. Validate this meets residency policy.
Email	Dedicated mailbox via Graph/IMAP; outbound via SMTP relay	Assistant replies in thread; sending to external recipients is approval-gated.
Public API and SDK	REST + Server-Sent Events, OpenAPI spec; OpenAI-compatible 'responses/chat' facade for agents; Python/TypeScript SDK	For internal applications embedding agents; service identities with scoped permissions; webhooks for run completion.

4.10 Identity, authorisation and governance

Authentication and identity propagation

- **SSO** via the corporate IdP (Entra ID, Okta, Ping, AD FS) using OIDC (preferred) or SAML, brokered by **Keycloak** (or the IdP directly). MFA and conditional access enforced by the IdP; SCIM for user and group provisioning and deprovisioning.
- **Token exchange** (OAuth 2.0 Token Exchange, RFC 8693, or IdP on-behalf-of flows) converts the user's session token into **downstream, audience-restricted, short-lived tokens** for each tool, so source systems see the real user and enforce their own permissions.
- **Per-user delegated credentials**: where a system needs user consent (e.g. mailbox access), users connect it once via OAuth; refresh tokens are stored encrypted in **HashiCorp Vault** (or OpenBao) under a per-user path and never exposed to the model or sandbox. Vault dynamic secrets issue short-lived database credentials where pass-through is unavailable.
- **Workload identity**: services authenticate with SPIFFE/SPIRE or Kubernetes service-account tokens and mTLS (service mesh such as Istio or Linkerd).

Authorisation model

- **RBAC** for platform roles: user, agent builder, tool owner, tenant admin, platform admin, auditor, approver.
- **ABAC** for runtime decisions combining user attributes (department, location, clearance), agent attributes (risk tier), tool risk class, data classification and context (channel, time, routine vs interactive). Policies as code in **OPA (Rego)** or **Cedar**, versioned in Git, unit-tested, evaluated by the orchestrator and MCP gateway on every call.
- **Effective permission = user entitlements AND agent allowlist AND policy.** An agent can never exceed the user's own rights; agent configurations can only narrow them.

Human-in-the-loop approvals

- Triggered by policy for W2/W3 tools, high-cost operations, unusual volumes (e.g. email to more than N recipients, updates touching more than N rows) or low-confidence classifications.
- The approval card shows the **exact arguments**, a human-readable preview (rendered email, SQL diff, list of affected records) and the agent's justification. Approval binds to a **hash of the arguments**; any change requires a new approval.
- Approvers: the requesting user by default; a designated owner or second person (four-eyes) for W3; step-up MFA for destructive actions. Timeouts default to deny.
- Users can grant **scoped standing approvals** (e.g. 'this routine may post to #team-digest') which are visible, revocable and expire.

Prompt-injection defences

Any content an agent reads (documents, emails, web pages, tickets, database rows, tool output) may contain instructions written by an attacker. Defences are layered because no single technique is complete:

1. **Treat tool output as untrusted data**: wrap results in clearly delimited, labelled blocks; the system prompt states that content inside them is data, never instructions; strip or neutralise control sequences and hidden text (zero-width characters, HTML comments, white-on-white text).
2. **Authority comes only from the user and policy**: tool output can never grant permissions, change recipients, add tools or approve actions. Approval cards make injected actions visible to humans.
3. **Capability containment**: least-privilege tool sets per agent; separate 'reader' subagents with no write tools to process untrusted content (dual-agent / quarantine pattern); no write tools in agents that browse the open web.
4. **Exfiltration controls**: egress allowlists; block rendering of model-generated external image/URL links that could leak data; outbound messages to new external recipients always approval-gated.
5. **Detection**: injection classifiers on retrieved content and tool output (flag, do not silently drop), anomaly detection on tool-call sequences, canary tokens in sensitive collections.
6. **Continuous red-teaming** with an injection test corpus in the evaluation suite (4.11).

PII, data protection and audit

- **PII detection and redaction** (e.g. Microsoft Presidio with custom recognisers for Aadhaar, PAN, bank account and phone formats) applied to logs, traces, memory writes and evaluation datasets; optional masking of PII before prompts for specific agents.
- **Data classification** labels flow from sources into chunks, messages and artefacts and drive ABAC, sandbox egress and retention.
- **Audit log**: append-only, hash-chained events (actor, on-behalf-of user, agent, tool, arguments hash, decision, approver, result summary, data classification), written to a write-once store (object lock) and streamed to the SIEM. Separate access for auditors; administrators cannot alter it.
- **Governance processes**: agent and tool review board for W2/W3 capabilities; model onboarding checklist (licence, evaluation, safety); periodic access reviews; incident response runbooks for agent misbehaviour (kill switch, revoke tokens, notify).

4.11 Observability and evaluation

Concern	Recommended tooling	What to capture
Tracing	**OpenTelemetry** SDKs and Collector (GenAI semantic conventions); backend Grafana Tempo or Jaeger	Span per turn, model call, tool call, retrieval, sandbox execution, approval wait; propagate trace IDs through MCP calls.
LLM observability	**Langfuse** (self-hosted; MIT core) or Arize Phoenix	Prompts, completions, tool I/O (redacted), token counts, latency, cost, user feedback, prompt versions, dataset runs.
Metrics	Prometheus + Grafana; vLLM and DCGM exporters	TTFT, tokens/s, queue depth, KV-cache usage, GPU utilisation and memory, batch sizes, error rates, tool latency, approval latency.
Logs	Loki or OpenSearch; structured JSON with trace IDs	Service logs with PII redaction.
Cost metering	Gateway metering + GPU-seconds per namespace (e.g. OpenCost)	Tokens and GPU-time per tenant, team, agent, routine and user; showback/chargeback dashboards and budget alerts.

Concern	Recommended tooling	What to capture
Evaluation harness	Langfuse datasets, promptfoo, Inspect, DeepEval, RAGAS; tool-calling suites (e.g. BFCL-style, tau-bench-style, MCP-based benchmarks) adapted to internal tools	Task success, tool-selection accuracy, argument validity, groundedness and citation accuracy, SQL execution accuracy, refusal correctness, latency and cost per task.
Red-teaming	garak, PyRIT, promptfoo red-team plugins; internal adversarial corpus	Prompt injection via documents/web/email, data exfiltration attempts, privilege escalation, jailbreaks, harmful content.

- **Evaluation gates**: every change to a model, prompt, agent configuration or tool description runs the regression suite in CI; promotion requires no statistically significant regression on key metrics.
- **Golden datasets** per agent and domain, built from pilot conversations (with consent and redaction) and expert-written cases; refreshed quarterly.
- **Online evaluation**: user feedback (thumbs, corrections), LLM-as-judge sampling on the fast or reasoning tier with periodic human calibration, and drift monitoring.

5. Key Sequence Flows

5.1 User chat turn with tool calls

A user asks for a summary of last week's P1 incidents with an MTTR chart. The orchestrator recalls memory, calls the reasoning model, executes a ticketing tool with the user's delegated token, runs Python in the session sandbox to chart the data, and streams the answer with the chart attached.

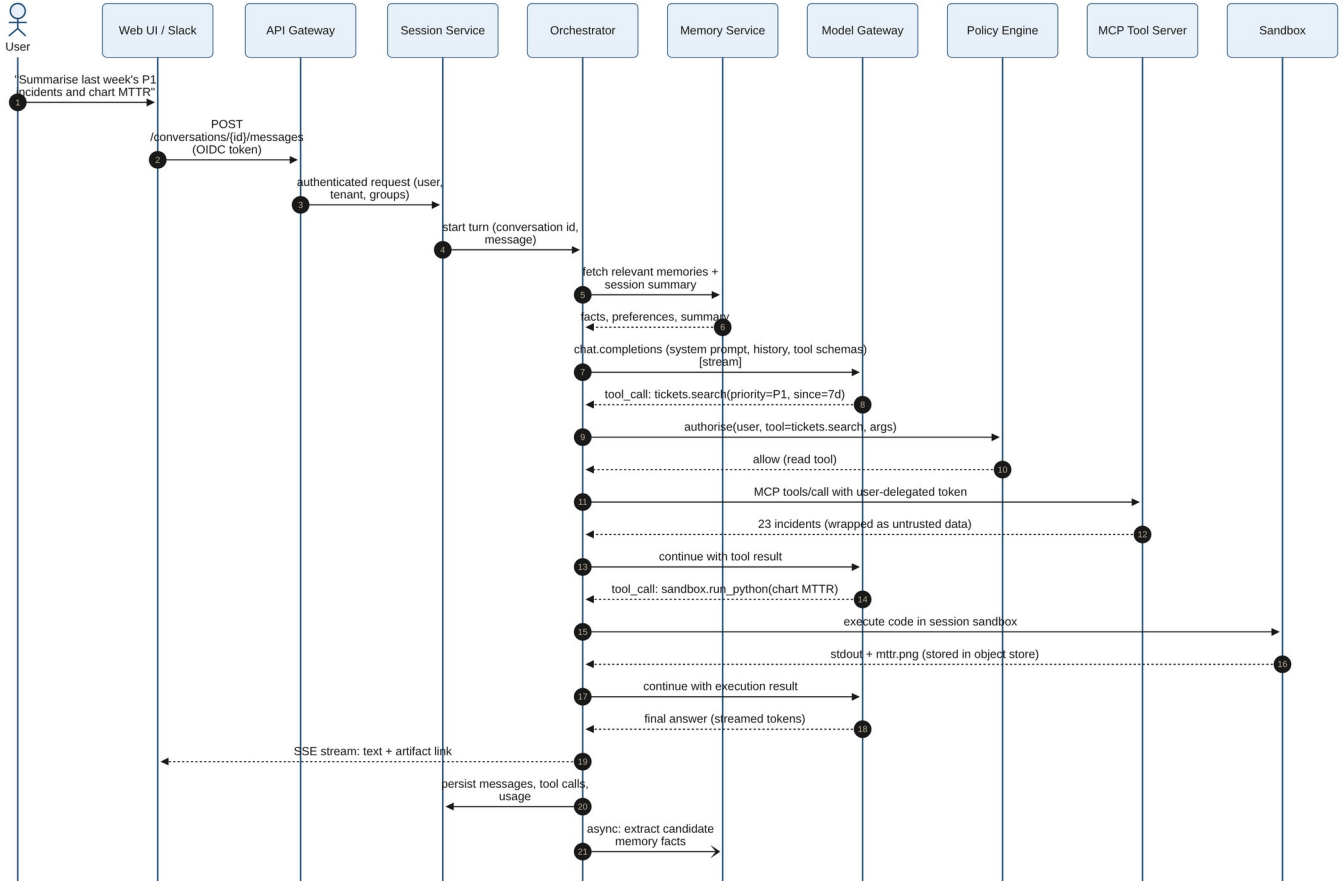


Figure 3: Chat turn with policy-checked tool calls and sandbox execution

5.2 RAG query with ACL filtering

The knowledge service resolves the caller's principals from the verified identity, applies them as a mandatory filter inside the hybrid search, reranks the candidates and, for sensitive collections, re-verifies permissions at the source before returning cited chunks.

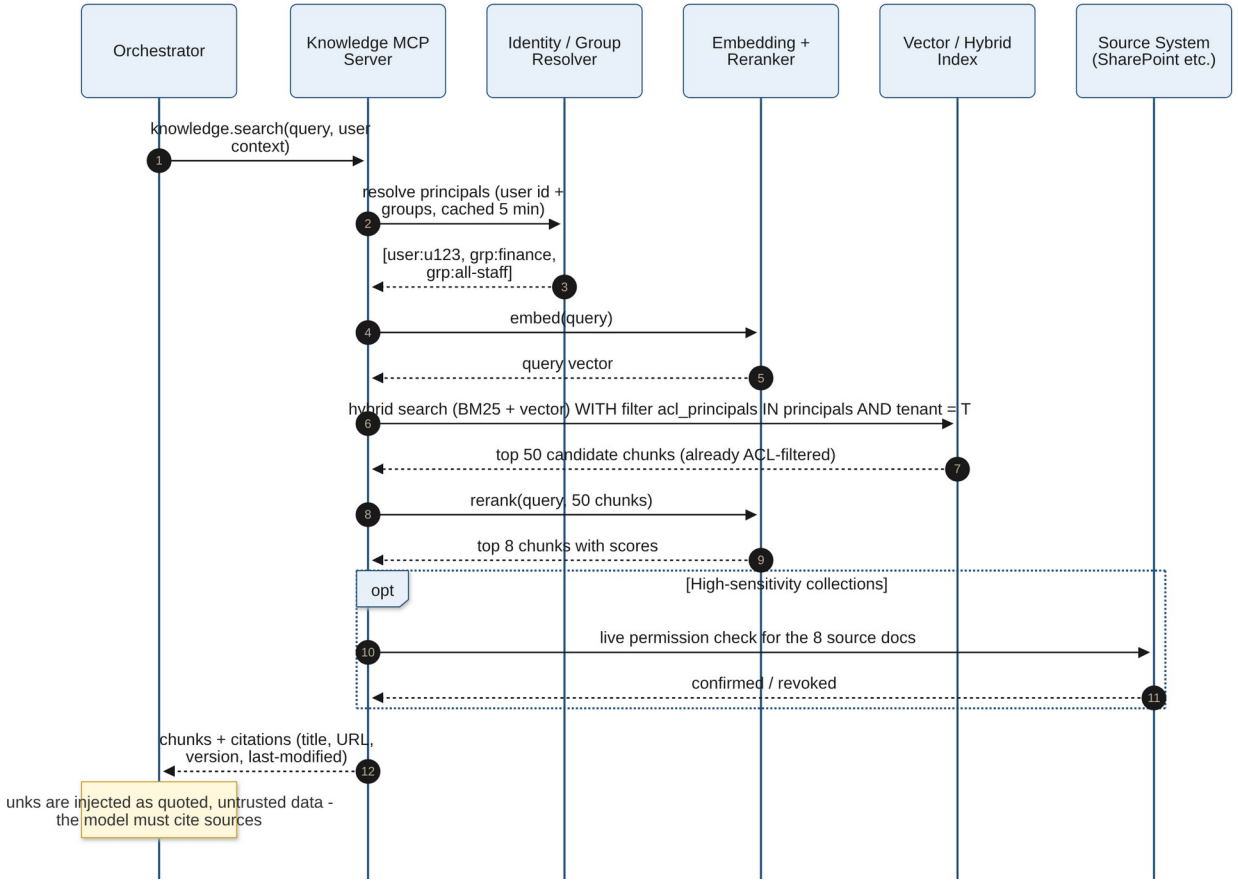


Figure 4: Retrieval with document-level ACL enforcement

5.3 Scheduled routine run

A Temporal Schedule (or an event) starts a routine workflow. The workflow obtains short-lived delegated credentials for the routine owner, runs the agent with a budget, and delivers results, either directly (pre-approved delivery) or after an approval.

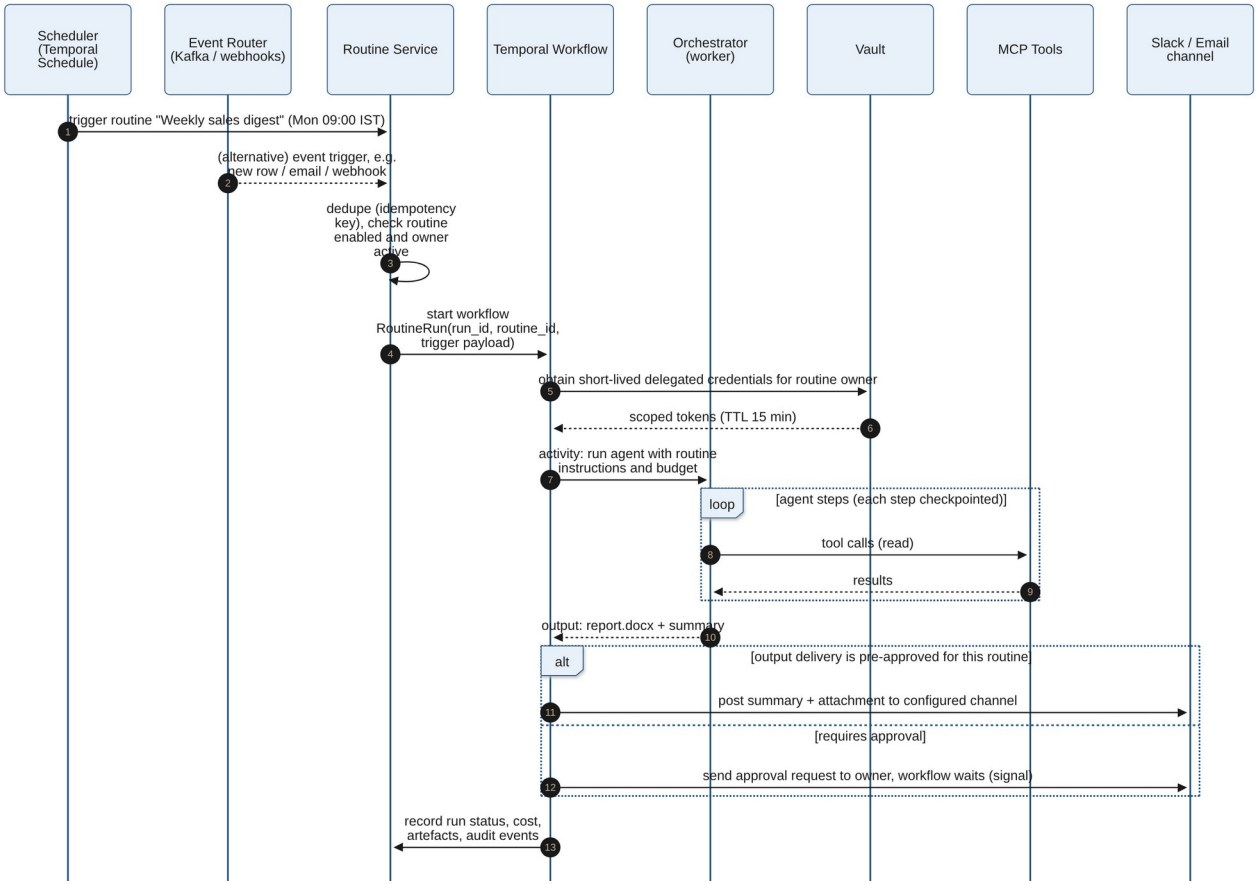


Figure 5: Scheduled or event-driven routine execution

5.4 Approval-gated write action

A policy decision returns require_approval for a production write. The workflow durably waits for a signal; the approver sees the exact arguments and preview; the execution verifies the argument hash before calling the tool, and both decision and outcome are audited.

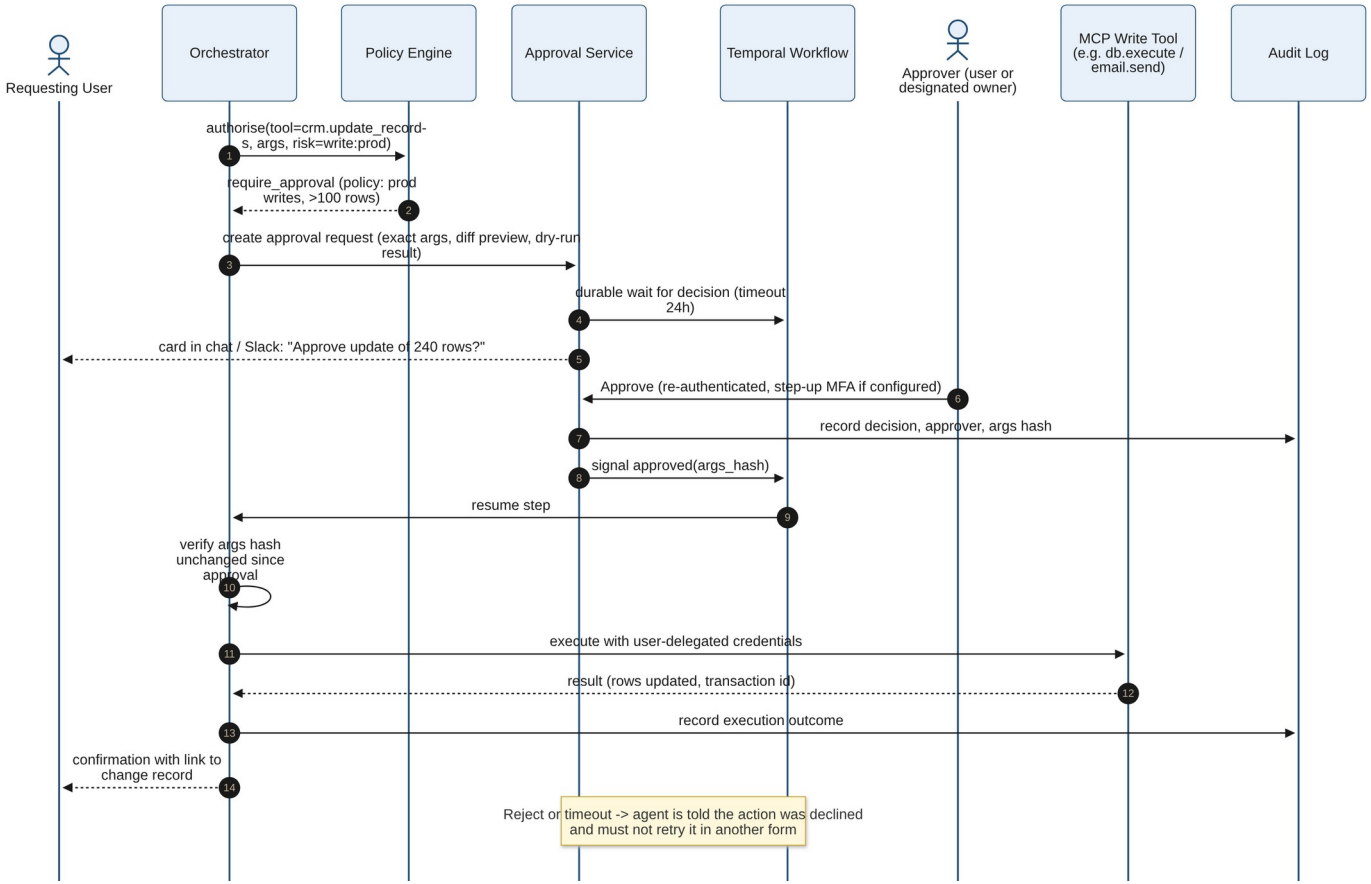


Figure 6: Human-in-the-loop approval for a risky write

6. Data Model Overview

The core relational model lives in PostgreSQL (one logical database per environment, tenant ID on every row with row-level security as defence in depth). Large payloads (tool results, files, screenshots) are stored in the object store and referenced by URI. Vector embeddings for memory live in pgvector; document chunks live in the RAG index.

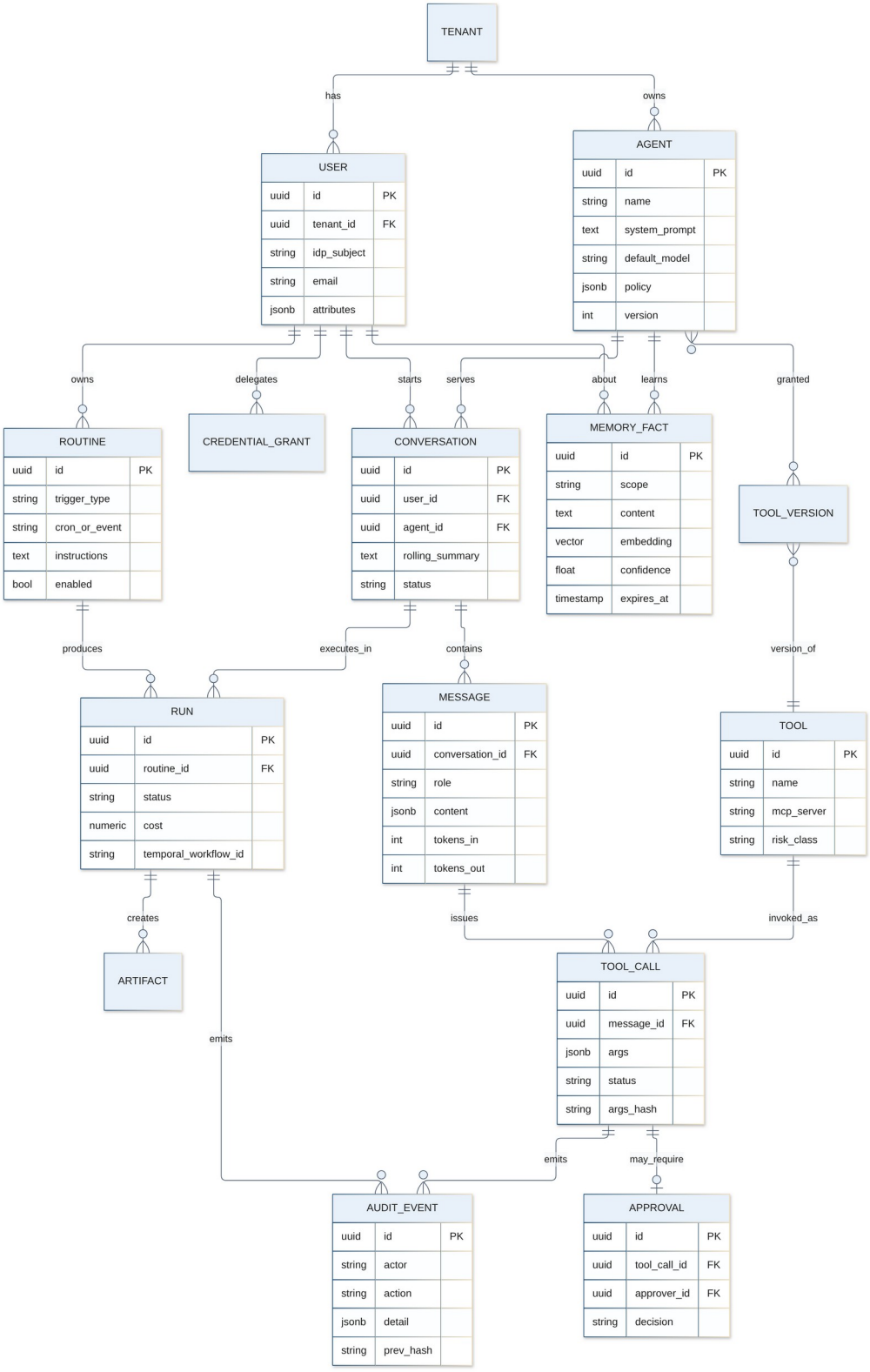


Figure 7: Core entities and relationships (simplified)

Entity	Purpose	Key attributes / notes
Tenant	Business unit or subsidiary boundary	Policies, quotas, encryption key reference, data residency tag.
User	Human principal (from IdP via SCIM)	IdP subject, email, groups (cached), attributes for ABAC, status.
Agent	Configured assistant	Versioned instructions, default model alias, tool grants (pinned versions), knowledge collections, policy, owner, risk tier.
Conversation	Thread between a user and an agent	Channel, rolling summary, status, retention class, sharing ACL.
Message	Turn content	Role, content parts (text, images, file refs), token usage, model, trace ID.
Tool / ToolVersion	Registered capability	MCP server, schema, description hash, risk class, scopes, owner, status.
ToolCall	One invocation	Arguments (redacted copy), args hash, policy decision, status, latency, result reference.
Approval	Human decision for a ToolCall	Approver, decision, timestamps, args hash, step-up flag, comment.
MemoryFact	Long-term memory item	Scope (user, agent, team), content, embedding, provenance (conversation, tool), trust level, confidence, expiry, superseded_by.
Routine	Saved triggered task	Trigger type and spec, instructions, agent, owner, delivery target, budget, standing approvals, enabled.
Run	One execution of a turn or routine	Status, Temporal workflow ID, cost, steps, artefacts, error.
Artifact	Generated file	Object URI, type, size, checksum, classification, retention.
CredentialGrant	User-connected integration	Provider, scopes, Vault path (no secret in DB), expiry, last used.
AuditEvent	Tamper-evident record	Actor, on-behalf-of, action, target, decision, detail, previous hash (chain).

7. Deployment Topology

The platform runs on Kubernetes on company hardware (bare metal or a private cloud such as OpenStack or VMware) or in a company-controlled private-cloud tenancy in an approved region. A distribution such as upstream Kubernetes with kubeadm/Cluster API, RKE2, OpenShift or a vendor-supported equivalent is acceptable; the design relies only on standard APIs.

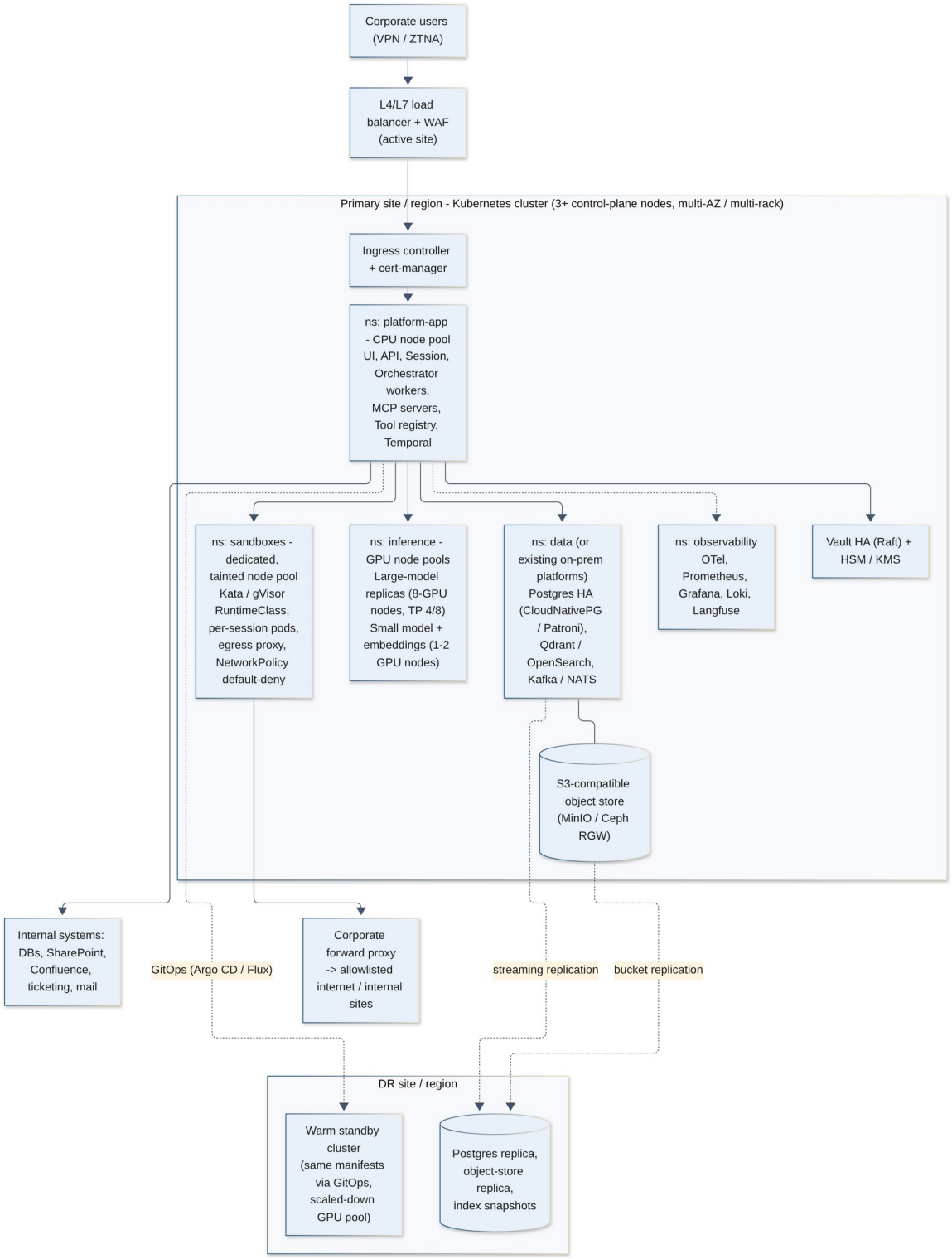


Figure 8: Deployment topology: primary site with warm DR site

7.1 Cluster layout and node pools

Pool	Hardware (indicative)	Workloads	Scaling
System / control plane	3 to 5 CPU nodes	Kubernetes control plane, ingress, mesh, monitoring agents	Static.
Platform app (CPU)	16 to 32 vCPU, 64 to 128 GB RAM nodes	UI, API, session, orchestrator workers, MCP servers, gateway, Temporal, ingestion workers	HPA on CPU and queue depth (KEDA on Temporal task-queue backlog / Kafka lag); cluster autoscaler where elastic.
Sandbox (CPU, isolated)	Bare-metal or nested-virt capable nodes, 32 to 64 cores	Kata/gVisor sandbox pods, browser workloads, egress proxy	Warm-pool size by time of day; scale on pending sandbox requests.
GPU inference	8-GPU servers (80 to 192 GB per GPU class) for large models; 1 to 4 smaller GPUs per node for fast tier and embeddings	vLLM/SGLang replicas, TEI, guardrail models	Replica autoscaling on queue depth / KV-cache utilisation; scale-to-min off-hours; model weights pre-cached on local NVMe.
Data	Storage-optimised nodes with NVMe, or existing database platforms	Postgres (CloudNativePG), Qdrant/OpenSearch, Kafka/NATS, MinIO	Vertical first; sharding for indexes in Phase 3.

- **GPU operations**: NVIDIA GPU Operator (or AMD GPU Operator) for drivers, device plugin, DCGM monitoring; MIG or time-slicing for small models; topology-aware scheduling for tensor parallel replicas; optional KServe, LLM-d style or Ray Serve controllers for model lifecycle, or plain Deployments managed by GitOps.
- **Model distribution**: weights stored in the internal registry (object store), pulled once per node to local NVMe via an init container or a node-level cache daemon; checksums verified.
- **Multi-node models** (very large MoE) need high-bandwidth interconnect (InfiniBand or RoCE) between GPU nodes; single-node models need only standard data-centre networking.

7.2 Networking

- Users reach the platform via corporate network/VPN/ZTNA only; Slack and Teams connectivity via Socket Mode or a narrowly scoped inbound relay in the DMZ.
- Service mesh with **mTLS** everywhere; Kubernetes NetworkPolicies default-deny per namespace; explicit allow rules (orchestrator to gateway, MCP gateway to MCP servers, MCP servers to their target systems).
- **Egress**: only the sandbox egress proxy and approved connectors may leave the cluster, via the corporate proxy; the inference namespace has **no egress**, which provably prevents calls to external model APIs.
- Private DNS and internal PKI (cert-manager with an internal CA); ingress TLS with corporate certificates; WAF in front of the API.

7.3 High availability

Component	HA approach
Stateless services	3+ replicas across failure domains (racks/AZs), PodDisruptionBudgets, rolling or blue/green deploys.
Model serving	At least N+1 replicas per model alias in production; gateway health checks and fallback to fast tier; staged model rollouts (canary by percentage of traffic).

Component	HA approach
Temporal	Multi-replica frontend/history/matching/worker services on HA Postgres (or Cassandra at scale); workflows resume on any worker.
PostgreSQL	CloudNativePG or Patroni with synchronous replica in-site and asynchronous replica to DR; PgBouncer pooling; PITR backups to object store.
Vector store	Qdrant replication factor 2+ with sharding; OpenSearch 3+ master-eligible nodes and replicas.
Object store	MinIO/Ceph erasure coding across nodes; versioning and object lock for audit buckets.
Vault	Integrated Raft storage, 3 or 5 nodes, auto-unseal with HSM/KMS.
Event bus	Kafka with replication factor 3 (KRaft) or NATS JetStream clustered.

7.4 Disaster recovery

- **Warm standby** at a second site/region: same manifests via GitOps (Argo CD or Flux), data services replicated (Postgres streaming replication, object bucket replication, vector index snapshots or re-index from source), GPU pool scaled to minimum (e.g. 25 to 50 percent capacity) to serve degraded mode after failover.
- **Targets**: RPO under 15 minutes for conversations, routines and audit; RTO under 4 hours full service, under 1 hour for chat in reduced capacity.
- Vector indexes are derived data; they can be rebuilt from sources if needed, but snapshotting reduces RTO.
- Quarterly DR drills; documented failover/failback runbooks; DNS or global load balancer switch.
- Backups: daily full + continuous WAL for Postgres; Vault snapshots; Temporal database backups; configuration in Git.

7.5 Environments

Separate **dev**, **staging** and **production** environments (at minimum separate namespaces with separate data and credentials; production on its own cluster). Staging mirrors production topology at small scale with synthetic or masked data and runs the evaluation suite on every release. A shared lower-environment GPU pool hosts smaller or quantised models to save capacity.

8. Security Threat Model

The table summarises principal threats using a STRIDE-informed view adapted to agentic systems (including categories highlighted in the OWASP Top 10 for LLM Applications and emerging agentic-AI guidance). Likelihood and residual risk should be assessed in a formal threat-modelling workshop.

#	Threat	Impact	Mitigations
T1	Indirect prompt injection via documents, emails, web pages or tool output	Agent performs unintended actions, leaks data	Untrusted-data wrapping; authority only from user/policy; approval of W2/W3 actions; quarantined reader subagents; egress allowlists; injection classifiers; red-team suite.
T2	Data exfiltration through tool calls, links, images or sandbox network	Confidential data leaves company	Default-deny egress; no external model APIs; block auto-rendered external URLs; DLP on outbound messages; approval for new external recipients; audit and anomaly detection.
T3	Privilege escalation via the agent (confused deputy)	User accesses data beyond their rights	Delegated per-user tokens; no shared super-user service accounts; ABAC policy on every call; ACL filters derived from verified identity, not model arguments.
T4	Sandbox escape or abuse (malware, crypto-mining, lateral movement)	Cluster compromise	Kata/Firecracker or gVisor; dedicated nodes; seccomp/AppArmor; no secrets in sandbox; network isolation; runtime detection (Falco/Tetragon); resource limits.
T5	Malicious or compromised MCP server / tool poisoning	Hidden instructions, data theft, wrong actions	Internal build and review of servers; pinned versions and signed images; description hash monitoring; gateway-enforced scopes; per-server network policies.
T6	Unsafe SQL or destructive writes	Data loss, outage, wrong decisions	Read replicas; read-only roles; parser-based validation; limits and timeouts; narrow write tools with approval, dry run and change windows.
T7	RAG ACL bypass or stale permissions	Exposure of restricted documents	In-query mandatory ACL filter; late verification for sensitive collections; fast permission sync; tombstone deletes; periodic ACL reconciliation tests.
T8	Memory poisoning	Persistent manipulation of future sessions	Provenance and trust levels on facts; no automatic promotion from untrusted sources; user-visible memory; review for agent-level memory.
T9	Model supply-chain risk (tampered weights, malicious code in model repos)	Backdoored behaviour, code execution at load	Download from vetted sources into quarantine; checksums and signatures; safetensors only (no pickle); disable remote code execution in loaders; evaluate before promotion.
T10	Sensitive data in logs, traces and eval datasets	Secondary data exposure	PII redaction in pipelines; access controls on observability tools; retention limits; encrypted storage.
T11	Denial of service / runaway costs (agent loops, huge prompts)	GPU starvation, outages	Per-user/agent quotas; step and token budgets; rate limits; circuit breakers on routines; priority classes (interactive over batch).
T12	Account takeover or stolen tokens	Impersonation	SSO with MFA; short-lived tokens; refresh tokens only in Vault; token binding to workload identity; anomaly alerts; immediate revocation via SCIM deprovisioning.
T13	Insider misuse by administrators	Unauthorised access to conversations	Separation of duties; break-glass with approval; access to conversation content audited; tamper-evident audit log outside admin control.
T14	Harmful, biased or incorrect output (hallucination)	Bad decisions, reputational or legal harm	Grounding with citations; answerability thresholds; output guardrail models; human review for high-stakes use cases; user education and disclaimers.
T15	Repudiation of actions	Inability to investigate incidents	Hash-chained audit with on-behalf-of identity, argument hashes and approvals; Temporal history; SIEM integration.

9. Phased Roadmap

Timelines assume a dedicated team, GPU hardware available at the start of each phase (procurement lead times can be months and should start during Phase 0), and timely security and legal reviews. Team sizes are full-time-equivalent engineers excluding part-time stakeholders.

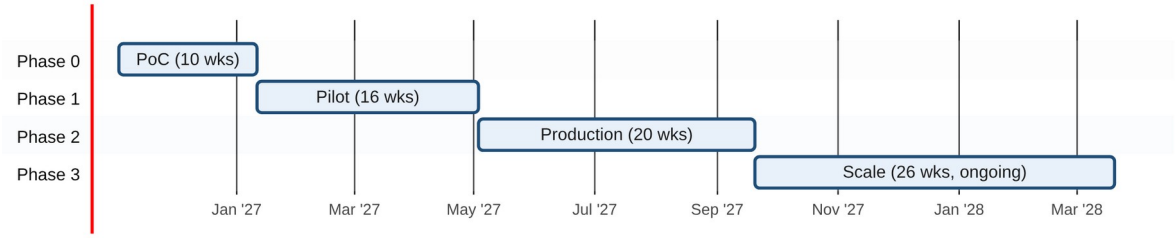


Figure 9: Indicative roadmap timeline

Phase	Scope	Exit criteria	Team (FTE)	Duration
Phase 0: PoC	One open-weight model on vLLM (2 to 4 GPUs) behind a LiteLLM-style gateway 2 to 3 MCP tools (e.g. read-only Postgres replica, Confluence or SharePoint search, Jira read) One sandbox type (gVisor pod) for Python and file generation Forked web UI (Open WebUI or LibreChat) with SSO Basic tracing in Langfuse; small eval set (50 to 100 tasks)	Demonstrated end-to-end agent tasks with 20 to 30 friendly users No external API calls (verified) Measured latency, task success and GPU throughput Go/no-go and hardware sizing for pilot	3 to 5 (platform/backend 2, ML/inference 1, infra 1, part-time security)	8 to 10 weeks
Phase 1: Pilot	Temporal-based orchestrator; reasoning + fast model tiers; embeddings/reranker RAG over 2 to 3 sources with ACL enforcement Policy engine, delegated tokens via Vault, approval flow Slack (or Teams) bot; memory v1; artefacts Evaluation suite in CI; red-team round 1	100 to 300 users across 2 to 3 departments Security review passed; DPIA completed Task success and satisfaction targets met for chosen use cases SLOs defined and measured	6 to 9 (+ security engineer, product manager, UX)	3 to 4 months
Phase 2: Production	HA across failure domains; DR site; production SLOs Routines (cron and events); tool registry with self-service onboarding Multi-tenant quotas and cost showback Expanded MCP catalogue (DB engines, ticketing, email, internal APIs) Audit to SIEM; runbooks; on-call	General availability for 1,000 to 5,000 users DR drill passed Audit and compliance sign-off Unit cost per active user tracked	10 to 14 (incl. SRE 2 to 3, security 1 to 2)	4 to 5 months
Phase 3: Scale	Additional GPU capacity and model specialisation (vision, coder, SQL LoRA) Multi-site active/active for chat; larger indexes (sharding) Agent marketplace and builder UX; advanced routing Evaluation-driven model upgrades each quarter	10,000+ users; many teams building agents Measured business outcomes per use case	12 to 18 (platform product team)	6 months, then ongoing

10. Indicative Cost Considerations

> **Important: estimates and assumptions only**
>
> The figures below are **rough planning estimates** derived from stated assumptions. They are not vendor quotations and contain no specific prices. Actual needs depend heavily on chosen models, quantisation, prompt sizes, agent step counts, usage patterns and hardware generation. Validate with Phase 0/1 telemetry and load tests before committing capital.

10.1 Sizing assumptions

- Peak concurrent **active** agent sessions are about 3 to 5 percent of named users (business-hours peak).
- An interactive agent turn averages 3 to 10 model calls, 8k to 20k input tokens per call (largely prefix-cached system prompt, tool schemas and history) and 300 to 800 output tokens per call.
- Reasoning tier: a 70B-dense-class or mid-size MoE model in FP8, one replica on 4 x 80 GB-class GPUs sustains roughly 30 to 60 concurrent sequences at acceptable latency (to be measured).
- Roughly 60 to 70 percent of model calls can be served by the fast tier after routing is tuned.
- Production adds N+1 redundancy; DR adds a warm standby at 25 to 50 percent of primary capacity.

10.2 Indicative GPU counts by user scale

Scale (named users)	Peak concurrent sessions (assumed)	Reasoning tier GPUs	Fast tier + embeddings + guardrails	Indicative total (primary site)	Notes
PoC (20 to 30)	1 to 3	2 to 4 (or 1 if a 4-bit / small-active MoE model)	Shared on same GPUs or 1 small GPU	2 to 4 GPUs	Can use existing or a single leased server.
Pilot (100 to 300)	5 to 15	4 (1 replica)	2 to 4	~8 GPUs (one 8-GPU server)	No redundancy; acceptable for pilot.
1,000	30 to 50	8 (2 replicas, N+1)	3 to 6	12 to 16 GPUs (2 servers)	Plus DR warm standby of ~4 to 8 GPUs.
5,000	150 to 250	20 to 32 (5 to 8 replicas)	8 to 12	32 to 48 GPUs (4 to 6 servers)	Routing and prefix caching strongly affect this range.
20,000	600 to 1,000	64 to 112	16 to 24	80 to 136 GPUs (10 to 17 servers)	Consider larger-memory GPUs, multi-node MoE and dedicated batch pools.

10.3 Hardware options

GPU class (examples)	Memory	Best suited for
NVIDIA H100 / H200 (SXM or NVL)	80 GB / 141 GB HBM	Reasoning tier with FP8; H200 memory allows larger batch and longer context per replica.
NVIDIA B200 / B300-class (Blackwell)	~180 to 288 GB HBM	Large MoE models and highest throughput (FP4/FP8); higher power and cooling requirements.
AMD Instinct MI300X / MI325X / MI350-class	192 to 288 GB HBM	Large models on fewer GPUs; supported by vLLM and SGLang (ROCm); validate tooling maturity for chosen models.

GPU class (examples)	Memory	Best suited for
NVIDIA L40S / L4, RTX PRO 6000-class	24 to 96 GB	Fast tier, embeddings, rerankers, vision models; PCIe servers in standard racks.
NVIDIA A100 (existing estate)	40 / 80 GB	Reuse for fast tier, embeddings and INT8/4-bit models (no FP8).

10.4 Cost components

Component	Considerations
GPU servers (capex or reserved private cloud)	Typically the dominant infrastructure cost; amortise over 3 to 5 years; consider phasing purchases with hardware generations.
Power, cooling and rack space	An 8-GPU high-end server can draw on the order of 10 kW or more; confirm data-centre power density and liquid-cooling needs for newest generations.
Networking	Standard 25 to 100 GbE for single-node models; InfiniBand/RoCE fabric only if multi-node models are needed.
CPU, storage and data services	App pools, sandbox pools (scale with concurrent sandboxes), NVMe for model caches, object storage for artefacts, audit and backups (multi-year retention).
Software and support	Most components are open source; budget for optional enterprise support (Kubernetes distribution, database, Vault, observability) and GPU software licences where applicable.
People	Platform, ML, SRE and security engineers (Section 9) usually exceed hardware cost in early phases.

Unit economics formula for internal comparison: cost per million tokens = (amortised hourly cost of a GPU replica + power and hosting) / (measured tokens per hour per replica x average utilisation). Track cost per active user per month and cost per completed task as the primary business metrics; utilisation (batching, routing to the fast tier, off-hours batch work such as ingestion and evaluations) is the main lever.

11. Risks, Open Questions and Next Steps

11.1 Key risks

Risk	Impact	Mitigation
Open-weight model quality is insufficient for some complex agentic tasks	Lower task success, user disappointment	Evaluate early on real tasks; use tools and retrieval to compensate; task decomposition with subagents; plan quarterly model upgrades; scope use cases to proven strengths.
GPU procurement lead times and capacity constraints	Delayed pilot or production	Start procurement during Phase 0; use quantisation and MoE to reduce needs; consider interim reserved private-cloud capacity within residency rules.
Security incident caused by agent actions (injection, over-permission)	Data breach, loss of trust	Layered defences (Section 4.10), approvals, conservative W2/W3 enablement, red-teaming before each phase.
Permission sync complexity across many sources	ACL leaks or missing results	Start with 2 to 3 well-understood sources; reconciliation tests; late verification for sensitive data.
Fast-moving open-source dependencies	Upgrade churn, breaking changes	Pin versions; internal mirrors; abstraction at gateway and MCP boundaries; dedicated upgrade cadence.
Adoption and change management	Low usage, poor ROI	Target high-value use cases with business sponsors; training; feedback loops; publish success metrics.
Regulatory changes (data protection, AI governance)	Rework	Engage legal and compliance early; DPIA; configurable retention, consent and audit.
Skills gap in GPU operations and LLM serving	Instability, poor utilisation	Hire or train ML-platform engineers; vendor support contracts; runbooks and load testing.

11.2 Open questions

1. Which 3 to 5 use cases (and business sponsors) define pilot success, and what are their measurable targets?
2. On-premises hardware or company-controlled private cloud for Phase 1? What GPU capacity, power density and lead times are available?
3. Which identity provider and directory are authoritative, and is OAuth token exchange / on-behalf-of supported for key systems (SharePoint, Exchange, Jira, databases)?
4. What data classification levels may the platform process in Phase 1 (e.g. Internal only vs Confidential/Restricted)?
5. Are Microsoft Teams bot connectivity requirements acceptable under the residency policy, or is Slack/web-only preferred?
6. Retention policy for conversations, memory, artefacts and audit logs; legal-hold requirements.
7. Licence approvals for candidate models (e.g. community licences with usage conditions vs Apache 2.0/MIT).
8. Build vs fork for the web UI, and which UI licence terms are acceptable for internal distribution.
9. Chargeback model: central funding vs per-team showback/chargeback.
10. Who owns MCP servers for each system of record: platform team or system owners?

11.3 Recommended next steps (next 30 to 60 days)

1. **Approve Phase 0** scope, team (3 to 5 engineers) and a 10-week timebox; nominate business sponsors and 2 to 3 pilot use cases.
2. **Secure PoC hardware** (2 to 4 data-centre GPUs) and **start pilot procurement** (one 8-GPU server class) in parallel given lead times.
3. Run a **model bake-off** on 100 representative internal tasks across 3 to 4 open-weight candidates in FP8 and 4-bit, measuring tool-call accuracy, SQL accuracy, latency and throughput.
4. Hold a **threat-modelling workshop** with security and data protection; agree risk classes, approval policy and data classification scope for the pilot.
5. Confirm **identity integration** patterns (OIDC, token exchange, SCIM) with the IAM team.
6. Stand up the **foundation**: Kubernetes namespaces, GitOps repo, Vault, Postgres, Langfuse, internal model registry and package mirrors.
7. Build the first **three MCP servers** with the shared safety core (read-only DB, document search, ticketing read) and the evaluation harness.
8. Define **success metrics and SLOs** for the pilot gate review.

Appendix A. Recommended Open-Source Stack Summary

Default recommendations with credible alternatives. Licences are indicative; confirm current licence terms at adoption, as several projects have changed licences in recent years.

Layer	Recommended default	Alternatives	Licence (indicative)
LLM inference	vLLM	SGLang, TGI, TensorRT-LLM on Triton	Apache 2.0
Embeddings / rerank serving	Hugging Face TEI	Infinity, Triton, vLLM pooling models	Apache 2.0
Model gateway	LiteLLM proxy	Envoy AI Gateway, Kong AI Gateway, custom proxy	MIT (core; enterprise features separate)
Open-weight LLMs	Qwen 3.x family, gpt-oss, Mistral (Apache 2.0 releases)	Llama 3.x/4, DeepSeek V3/V4, GLM, Gemma, Granite	Model-specific; review each
Embedding / reranker models	BGE-M3 + bge-reranker-v2-m3	Qwen3-Embedding / Reranker, multilingual-e5, nomic-embed	MIT / Apache 2.0 (verify)
Agent loop	Custom thin loop (Python or TypeScript) using OpenAI client + MCP SDK	LangGraph, PydanticAI, OpenAI Agents SDK (local endpoints), Microsoft Agent Framework	MIT / Apache 2.0
Durable execution	Temporal	Restate, DBOS, Hatchet, queue + state table	MIT
Tool protocol	MCP (official SDKs), streamable HTTP	OpenAPI-wrapped tools via MCP adapters	MIT
Policy engine	Open Policy Agent (Rego)	Cedar, OpenFGA (relationship-based)	Apache 2.0
Identity broker	Keycloak	Corporate IdP directly, Authentik, Dex	Apache 2.0
Secrets	HashiCorp Vault	OpenBao (open-source fork), cloud KMS-backed stores	BSL 1.1 (Vault) / MPL 2.0 (OpenBao)
Relational store	PostgreSQL (CloudNativePG operator)	Patroni, existing enterprise DB platform	PostgreSQL / Apache 2.0
Vector / hybrid search	Qdrant; pgvector for memory	OpenSearch, Milvus, Weaviate, Vespa	Apache 2.0 / PostgreSQL
Document parsing	Docling, Apache Tika	Unstructured, PaddleOCR, Tesseract	MIT / Apache 2.0
Object storage	MinIO or Ceph RGW	Existing S3-compatible enterprise storage	AGPLv3 (MinIO) / LGPL (Ceph)
Event bus / CDC	Kafka (KRaft) + Debezium	NATS JetStream, Redpanda, RabbitMQ	Apache 2.0
Sandbox isolation	Kata Containers; gVisor	Firecracker-based sandbox platforms	Apache 2.0
Browser automation	Playwright + headless Chromium	Puppeteer, Selenium	Apache 2.0
Web UI base	Open WebUI or LibreChat (fork)	Custom React / Next.js UI	Check current terms (branding clauses) / MIT
Chat bots	Slack Bolt; Bot Framework / Teams AI library	Mattermost or Rocket.Chat for fully self-hosted chat	MIT
Tracing / telemetry	OpenTelemetry + Grafana Tempo	Jaeger	Apache 2.0 / AGPLv3
LLM observability and evals	Langfuse	Arize Phoenix, MLflow tracing	MIT (core)
Metrics / logs / dashboards	Prometheus, Loki, Grafana	OpenSearch Dashboards, VictoriaMetrics	Apache 2.0 / AGPLv3
Evaluation and red-team	promptfoo, Inspect, RAGAS, DeepEval, garak, PyRIT	Custom harness	MIT / Apache 2.0
PII detection	Microsoft Presidio	spaCy/GLiNER-based custom recognisers	MIT

Layer	Recommended default	Alternatives	Licence (indicative)
Runtime security	Falco or Tetragon	Kubernetes audit + SIEM rules	Apache 2.0
GitOps / IaC	Argo CD, Terraform/OpenTofu, Helm	Flux, Crossplane	Apache 2.0 / MPL 2.0
GPU operations	NVIDIA / AMD GPU Operator, DCGM exporter	KServe, Ray Serve, llm-d	Apache 2.0

Appendix B. Glossary

Term	Meaning
ABAC / RBAC	Attribute-based / role-based access control.
Agent loop	Iterative cycle in which a model reasons, calls tools and observes results until the task is done.
Hybrid search	Combination of lexical (BM25/sparse) and semantic (dense vector) retrieval.
KV cache	Per-token attention state held in GPU memory during generation; dominant memory consumer at high concurrency.
MCP	Model Context Protocol: open standard for exposing tools, resources and prompts to AI applications.
MoE	Mixture-of-Experts: model architecture where only a subset of parameters is active per token.
Prefix caching	Reuse of computed KV cache for identical prompt prefixes across requests.
RLS	Row-level security in a database.
TTFT	Time to first token.
TP / EP	Tensor parallelism / expert parallelism across GPUs.